



eRecrutement

Documentation technique

Ce document a pour but de décrire l'analyse, la conception, et la réalisation du projet eRecrutement.

Auteur	Gabriel Noé
Numéro du candidat	151928
Date de création	7 mai 2025
Dernière édition	20 mai 2025

Table des matières

1. Résumé du rapport.....	4
2. Préambule.....	5
2.1. Introduction	5
2.2. Organisation	5
2.3. Environnement de travail	5
2.4. Déroulement du projet.....	6
2.5. Déclarations	8
3. Cahier des charges	9
4. Analyse	13
4.1. État initial	13
4.2. État désiré	13
4.3. Public cible	14
4.4. Besoins.....	14
4.4.1. Cas d'utilisation	14
4.4.2. Description.....	16
4.5. Fonctionnement	19
4.5.1. Système d'authentification	19
4.5.2. Formulaire.....	21
4.5.3. Administration	22
4.6. Ébauches conceptuelles	26
4.6.1. Consulter les fichiers d'une candidature	29
4.7. Sécurité et risques	30
4.7.1. Upload des fichiers	30
4.7.2. Envoi du formulaire	32
4.8. Technologie utilisées.....	33
5. Conception	34

5.1.	Schéma d'architecture.....	34
5.2.	Diagrammes de séquences.....	35
5.3.	Base de données.....	44
5.4.	Stratégie de tests	50
5.4.1.	Formulaire.....	50
5.4.2.	Interface administrateur	51
6.	Réalisation.....	52
6.1.	Structure du projet.....	52
6.2.	Gestion des fichiers attachés aux candidatures	54
6.3.	Authentification	58
6.4.	Gestion des commentaires et des notes.....	61
6.5.	Coding styles	62
6.6.	Validation des données du formulaire	63
6.7.	Base de données.....	65
6.8.	Traitement des erreurs	66
6.9.	Style et design avec Vuetify.....	69
6.10.	Gestion des langues	69
6.11.	Gestion des configurations	70
7.	Tests de fonctionnement.....	71
8.	Problème rencontré.....	72
9.	Amélioration et évolution	73
10.	Conclusion	74
11.	Remerciements	75
12.	Déclaration de non plagiat	75
13.	Glossaire.....	76
14.	Annexes	77
15.	Table des illustrations.....	78

1. Résumé du rapport

Situation de départ :

Le Centre d'Apprentissage TIC (CA TIC) de la HEIA-FR reçoit chaque année un grand nombre de candidatures pour les places d'apprentissage en informatique. Jusqu'à présent, ces candidatures étaient gérées à l'aide du système web général de la HES-SO//FR, qui s'avérait peu adapté aux besoins spécifiques du recrutement d'apprentis, en particulier pour des jeunes en sortie d'école obligatoire. Ce système ne permettait pas une gestion ergonomique des dossiers de candidature et ne facilitait ni la consultation rapide des pièces jointes, ni la notation, ni les échanges entre formateurs sous forme de commentaires.

Mise en œuvre :

Le projet « eRecrutement » a été conçu pour répondre à ces problématiques. Une application web a été développée en utilisant Node.js, Express, Vue.js et MySQL, selon une architecture en trois tiers.

Elle se compose de deux interfaces :

- Un formulaire public permettant aux candidats de déposer leur dossier, avec validation dynamique, upload de fichiers et interface responsive.
- Une interface privée réservée aux administrateurs, protégée par une authentification Microsoft via la « CA TIC Authentication Gateway », permettant de visualiser, commenter, noter et trier les candidatures.

Des règles de sécurité ont été mises en place, notamment sur les fichiers téléversés (type, taille, nom, répertoire protégé, renommage). Une validation stricte est appliquée à tous les champs, côté client et côté serveur.

Chaque fonctionnalité a été testée indépendamment tout au long du développement : authentification, formulaire, consultation des fichiers, gestion des commentaires, gestion des notes, tri des candidatures, affichage de toutes les candidatures et affichage des détails d'une candidature.

Résultats :

Toutes les fonctionnalités ont été implémentées et validées. L'application permet désormais aux candidats de soumettre facilement leur dossier depuis n'importe quel appareil. Du côté des administrateurs :

- La consultation des candidatures est fluide et rapide.
- Les commentaires et les notes sont propres à chaque évaluateur et affichés de manière claire.
- La moyenne des notes est calculée automatiquement.
- Les documents sont consultables directement via le navigateur, dans un nouvel onglet.

L'authentification centralisée garantit une sécurité renforcée, et la base de données relationnelle assure une bonne traçabilité ainsi qu'une intégrité des données. Le système s'avère ainsi plus adapté, sécurisé et efficace que le processus précédent, avec une nette amélioration de l'expérience utilisateur pour les deux types d'utilisateurs : candidats et recruteurs.

2. Préambule

2.1. Introduction

Le projet eRecrutement est un travail de CFC réalisé dans le cadre de la formation d'informaticien d'entreprise à la Haute école d'ingénierie et d'architecture de Fribourg (HEIA-FR). Il vise à développer une plateforme web pour la gestion des dossiers de candidature aux postes d'apprentis en informatique du Centre d'Apprentissage TIC (CA TIC) de la HEIA-FR. Ce document propose un suivi détaillé de l'avancement du projet.

2.2. Organisation

Candidat : Noé Mathys Gabriel, HEIA-FR

Chef de projet : Joël Dacomo, HEIA-FR

Expert principal : Mario Ramalho, EMF

Expert secondaire : Marc Stritt, Promed Laboratoire médical SA et EMF

2.3. Environnement de travail

Matériel

Les machines suivantes sont à disposition :

- MacBook Pro 16-inch, 2019, macOS Sonoma 14.4.1
- PC portable HP, Windows 11

Logiciels et systèmes

Les logiciels et outils suivants sont utilisés :

- Draw.io
- Git
- Gitlab
- IntelliJ IDEA
- PlantUML
- SequenceDiagram.org
- Suite Microsoft Office M365
- ChatGPT

Accès

Les accès suivants sont accordés :

- Réseau de la HEIA-FR
- Passerelle d'authentification Microsoft « CA TIC Authentication Gateway »

2.4. Déroulement du projet

Méthode de travail

Pour ce projet, une version adaptée de la méthode waterfall (méthode en cascade) a été choisie. Cette approche progresse de manière linéaire en suivant les étapes d'analyse, de conception, de réalisation, puis de tests.

L'adaptation principale consiste à effectuer des tests à la fin de chaque fonctionnalité développée, plutôt qu'une phase unique de tests en fin de projet. Cette organisation permet de garantir que chaque composant est pleinement opérationnel avant de passer au suivant, améliorant ainsi la qualité et la stabilité du système tout au long du cycle de développement.

Sauvegarde des données

Les documents produits au cours du projet sont enregistrés quotidiennement sur les disques réseau de la HEIA-FR, lesquels sont sauvegardés conformément à la politique de sauvegarde du service informatique de la HES-SO//Fribourg.

Source : <https://sinfo.hefr.ch/fr/catalogue-de-services/securite-et-protection-des-donnees/sauvegarde-des-donnees/>

Journal de bord

Un journal de bord est tenu pour suivre, jour après jour, l'avancement du projet, les difficultés rencontrées et les résultats obtenus. Couvrant l'intégralité de la durée du projet, il consigne quotidiennement les tâches réalisées, les objectifs atteints, les obstacles identifiés ainsi que les pistes et solutions explorées. Le journal de bord complet figure en annexe de ce rapport.

Glossaire

Un lexique figure en annexe de ce rapport ; il fournit les définitions des termes techniques et des acronymes employés.

Planification

Le projet est planifié à l'aide d'un diagramme de Gantt. Deux versions sont élaborées : la planification de référence et la planification en cours, afin d'illustrer l'évolution du projet. Le diagramme de planification est présenté ci-dessous.

Tâches	Mercredi 7 mai 25								Jeudi 8 mai 25								Vendredi 9 mai 25								Lundi 12 mai 25								Mardi 13 mai 25							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Administratif																																								
Démarrage du projet	x																																							
Planification du projet		x	x																																					
Réunion avec le chef de projet						x																																		
1ère visite des experts									x																															
2ème visite des experts																																								
Analyse																																								
Analyse des besoins, des fonctionnalités				x	x		x	x	x		x	x	x																											
Ebauches conceptuelles														x	x	x																								
Conception																																								
Schéma UML																x	x	x																						
Schéma de base de données																			x	x	x																			
Stratégie de tests																																								
Réalisation																																								
Mise en place du projet (Git, Init. code, Coding style)																																								
Formulaire de postulation																																								
Affichage des candidatures																																								
Affichage des détails d'une candidature																																								
Consultation d'un document d'une candidature																																								
Tri des candidatures																																								
Système d'authentification																																								
Système de notation et de commentaires																																								
Tests de fonctionnement																																								
Tests de l'application selon la stratégie																																								
Documentation																																								
Documentation technique																																								
Finalisation des documents et envoi du projet																																								

Figure 1 - Planification du 7 au 13 mai

Tâches	Mercredi 14 mai 25								Jeudi 15 mai 25								Vendredi 16 mai 25								Lundi 19 mai 25								Mardi 20 mai 25							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Administratif																																								
Démarrage du projet																																								
Planification du projet																																								
Réunion avec le chef de projet																																								
1ère visite des experts																																								
2ème visite des experts																																								
Analyse																																								
Analyse des besoins, des fonctionnalités																																								
Ebauches conceptuelles																																								
Conception																																								
Schéma UML																																								
Schéma de base de données																																								
Stratégie de tests																																								
Réalisation																																								
Mise en place du projet (Git, Init. code, Coding style)																																								
Formulaire de postulation																																								
Affichage des candidatures																																								
Affichage des détails d'une candidature																																								
Consultation d'un document d'une candidature																																								
Tri des candidatures																																								
Système d'authentification																																								
Système de notation et de commentaires																																								
Tests de fonctionnement																																								
Tests de l'application selon la stratégie																																								
Documentation																																								
Documentation technique																																								
Finalisation des documents et envoi du projet																																								

Figure 2 - Planification du 14 au 20 mai

2.5. Déclarations

Connaissances préalables

Le candidat possède les compétences techniques suivantes :

- Node.js
- Vue
- Express
- REST / RESTful
- MySQL / SQL
- Git

Utilisation des modèles d'entreprise

Les modèles d'entreprises suivants sont utilisés :

- Documentation technique et journal de bord (accessible par l'intranet)
- Conventions pour VCS (Version Control System, Git) :
 - o <https://nvie.com/posts/a-successful-git-branching-model>
 - o <https://api-ti.pages.forge.hefr.ch/a05/git/#message-de-commit>
- Conventions de codages :
 - o HTML: <https://api-ti.pages.forge.hefr.ch/a07/code-style>
 - o CSS : <https://api-ti.pages.forge.hefr.ch/a08/code-style>
 - o SQL : <https://api-ti.pages.forge.hefr.ch/a12/sql/#convention-de-nommage>

Intelligence artificielle

Des outils d'intelligence artificielle ont été utilisés pour améliorer la qualité rédactionnelle de ce rapport. Ils ont notamment servi à la correction orthographique, à la reformulation grammaticale. Il ne s'agit en aucun cas d'un remplacement des compétences du candidat, mais d'un outil d'assistance visant à renforcer la qualité de la documentation.

3. Cahier des charges

Ce chapitre décrit la consigne du travail telle qu'elle a été communiquée au début du projet dans le cahier des charges.

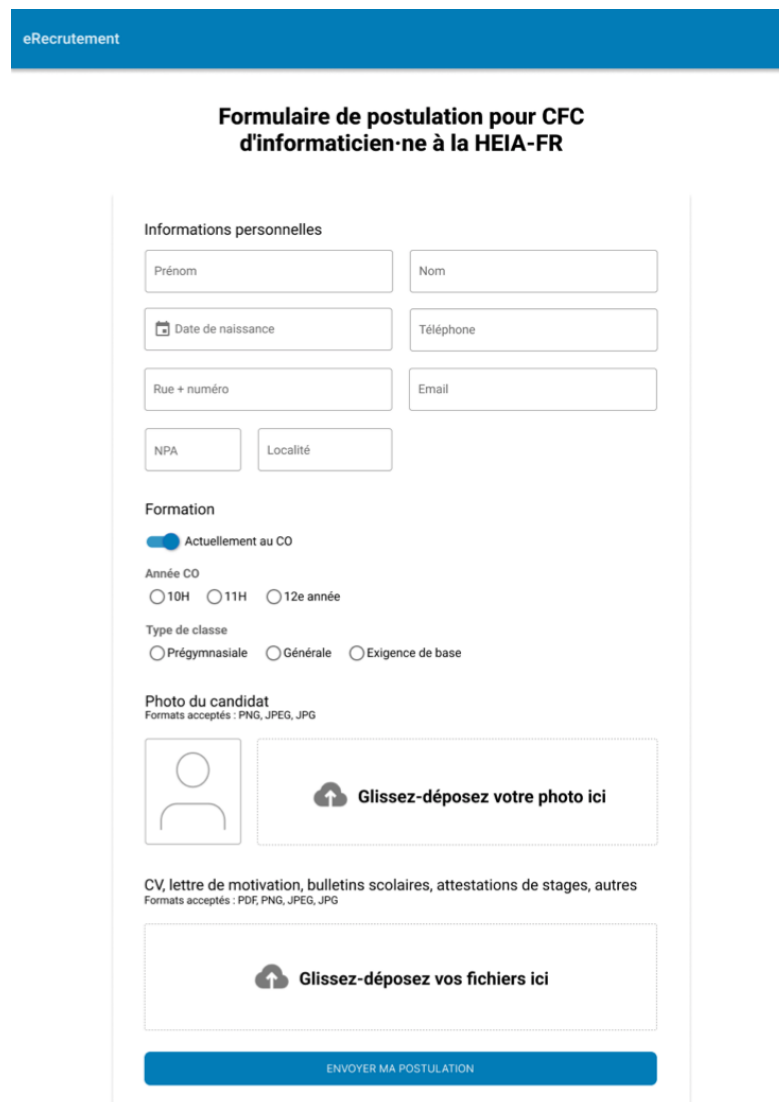
Donnée du problème

« Le système d'information, basé web, utilisé actuellement par la HES-SO//FR pour la gestion des offres d'emploi ne satisfait pas aux besoins pour le recrutement des apprentis en informatique du Centre d'Apprentissage TIC de la HEIA-FR.

Le but est de créer une application web moderne dédiée spécifiquement à la gestion des dossiers de candidature pour les postes d'apprentis en informatique. »

Description et objectifs du projet

« Un formulaire web permettra de déposer un dossier de candidature. Le formulaire devra être analysé et conçu sur la base des mockups ci-dessous :



The mockup shows a web form titled "Formulaire de postulation pour CFC d'informaticien-ne à la HEIA-FR". It is part of the "eRecrutement" system. The form is divided into several sections:

- Informations personnelles:** Includes input fields for Prénom, Nom, Date de naissance, Téléphone, Rue + numéro, Email, NPA, and Localité.
- Formation:** Features a toggle switch for "Actuellement au CO", a radio button selection for "Année CO" (10H, 11H, 12e année), and radio button selection for "Type de classe" (Prégymnasiale, Générale, Exigence de base).
- Photo du candidat:** Includes a placeholder for a photo and a button to upload it, with the text "Glissez-déposez votre photo ici".
- CV, lettre de motivation, bulletins scolaires, attestations de stages, autres:** Includes a placeholder for documents and a button to upload them, with the text "Glissez-déposez vos fichiers ici".

At the bottom of the form is a blue button labeled "ENVOYER MA POSTULATION".

Figure 3 - Cahier des charges: Mockup de formulaire de postulation

Le formulaire permettant l'upload de fichiers, une attention particulière devra être faite sur le transfert de ces données ainsi que sur les risques de sécurité inhérents à ce type d'opération. Ces aspects devront donc être analysés en détail et documentés en conséquence.

Le design devra être responsif pour tous types d'appareils et donc pleinement utilisable sur smartphone ou tablette tactile.

La validation des données doit être effectuée du côté frontend et backend avec un traitement approprié et clair pour les utilisateurs en cas de problèmes. Un jeu de tests, pouvant être exécuté dans Postman, permettra de démontrer le bon fonctionnement de la validation des données côté backend. Le jeu de tests contiendra des ensembles de données valides et invalides permettant de prouver que toutes les spécifications des champs de formulaires sont effectivement implémentées dans la validation des données du formulaire dans le backend.

Un système d'authentification permettra à un ensemble défini de personnes d'accéder à la liste des postulations.

La liste des postulations devra être analysée et conçue sur la base du mockup ci-dessous :

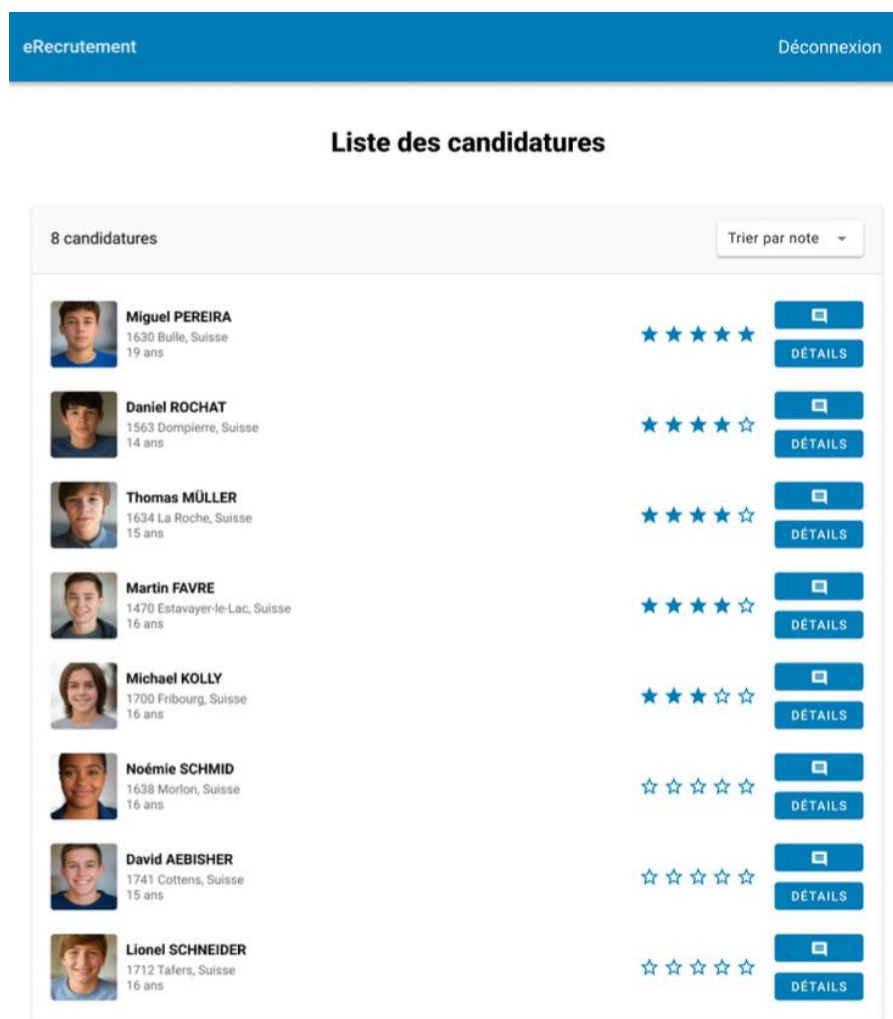


Figure 4 - Cahier des charges : Mockup de la liste des candidatures

Le design devra être responsif pour tous types d'appareils et donc pleinement utilisable sur smartphone ou tablette tactile.

Il devra être possible de trier la liste selon certains critères tels que : évaluation notée, nom de famille et date de réception du dossier.

Un bouton permettra d'accéder aux commentaires et évaluations notées de chaque entrée de la liste. L'affichage de ces informations se fera sous forme de fenêtre overlay ou de popup en se basant sur le mockup ci-dessous :

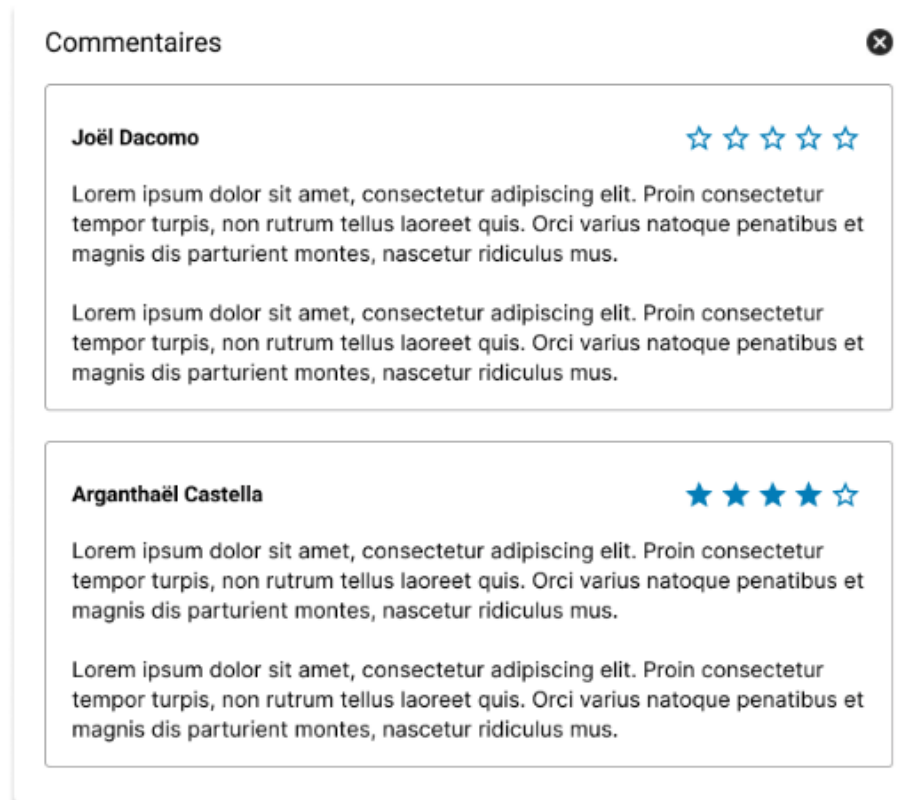


Figure 5 - Cahier des charges: Mockup de la fenêtre des commentaires

Finalement, un bouton Détails permettra d'accéder à une page permettant de consulter un dossier de candidature. La page de détails devra être analysée et conçue sur la base du mockup ci-dessous :

eRecrutement

Liste des candidaturesDéconnexion



★★★★☆
Thomas MÜLLER
thomas.muller@example.com
076 521 07 12
15 ans

Formulaire envoyé le 21.01.2025

Formation

Actuellement au CO: Oui
Type de classe: Prégymnasiale
Année: 11H

Adresse

Impasse de l'exemple 42
1634 La Roche
Suisse

Documents

 CV.pdf

 Lettre de motivation.pdf

 Bulletins scolaires.pdf

 Attestation de stage HEIA-FR.pdf

Notes, commentaires

Joël Dacomo★★★★☆

Commentaire

Publier

Arganthaël Castella★★★★☆

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin consectetur tempor turpis, non rutrum tellus laoreet quis. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin consectetur tempor turpis, non rutrum tellus laoreet quis. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus.

Figure 6 - Cahier des charges: Mockup de la page de détails d'une candidature

Le design devra être responsif pour tous types d'appareils et donc pleinement utilisable sur smartphone ou tablette tactile. Sur cette page, on pourra consulter l'ensemble des informations du dossier de candidature. Il sera possible de télécharger les documents qui ont été joints depuis le formulaire web. Chaque évaluateur pourra ajouter et éditer son propre commentaire uniquement. Il pourra également évaluer le dossier grâce à un système de notation basé sur un nombre d'étoiles allant de 1 à 5. Le nombre d'étoiles au-dessus du nom du candidat correspond à la moyenne arrondie à la valeur supérieure de l'ensemble des évaluations publiées. »

4. Analyse

4.1. État initial

Le Centre d'Apprentissage TIC (CA TIC) de la HEIA-FR reçoit chaque année plus de 80 candidatures pour le poste d'apprenti en informatique.

Actuellement, les responsables du CA TIC utilisent un système de gestion des postulations employé par la HES-SO//FR pour toutes les autres offres d'emploi. Le problème est que ce système ne répond pas aux besoins spécifiques du CA TIC pour le recrutement d'apprentis, qui sortent souvent de l'école obligatoire.

4.2. État désiré

Pour répondre à cette problématique, le CA TIC souhaite développer une plateforme web permettant aux candidats de postuler via un formulaire en ligne, ainsi qu'une interface réservée aux administrateurs (responsables du recrutement du CA TIC), afin de visualiser les informations de chaque candidature et d'y ajouter un commentaire ainsi qu'une note individuelle par administrateur.

Le formulaire inclura :

- les informations personnelles
- les coordonnées
- la formation (scolarité obligatoire ou non)
- les fichiers requis
- une photo du candidat.

L'interface d'administration devra inclure :

- une page listant toutes les candidatures,
- des fonctionnalités de tri par note, nom de famille et date de réception du dossier,
- un bouton permettant d'afficher rapidement les commentaires et notes de chaque administrateur dans une fenêtre superposée (overlay),
- un second bouton permettant d'afficher les détails complets de la candidature.

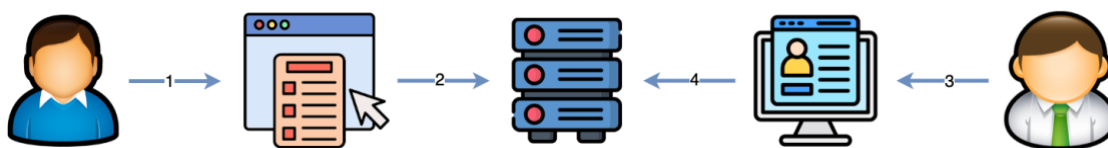


Figure 7 - Principe de fonctionnement

Explications : Les utilisateurs remplissent le formulaire et l'envoient. Les données sont ensuite traitées, enregistrées dans la base de données, et les fichiers attachés sont stockés sur le serveur. Les administrateurs accèdent à l'interface pour consulter les candidatures, y ajouter des notes et des commentaires. Ces informations sont ensuite traitées et enregistrées dans la base de données.

4.3. Public cible

Le formulaire est destiné à tous les candidats potentiels aux places d'apprentissage du Centre d'Apprentissage TIC (CA TIC) de la HEIA-FR :

- Les jeunes de 14 à 16 ans actuellement au cycle d'orientation (CO)
- Les personnes plus âgées en reconversion professionnelle

Les candidats peuvent avoir un numéro de téléphone ou une adresse postale à l'étranger, le formulaire devra donc être adapté à ces cas. Les administrateurs sont des formateurs professionnels et recruteurs. L'application doit être moderne et intuitive afin de pouvoir être utilisée sans guide ni formation préalable.

4.4. Besoins

Dans cette section, les besoins de l'application sont identifiés et analysés.

4.4.1. Cas d'utilisation

Légendes

Un acteur dans un cas d'utilisation est une entité (humaine ou non) qui interagit avec le système.

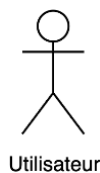


Figure 8 - Représentation UML d'un acteur

Un cas d'utilisation représente une fonctionnalité de l'application.

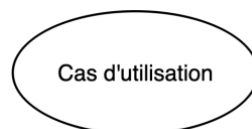


Figure 9 - Représentation UML d'un cas d'utilisation

Une association permet de lier un acteur à un cas d'utilisation.



Figure 10 - Représentation UML d'une association

Une relation « include » est utilisée pour indiquer que le cas d'utilisation source inclut systématiquement un autre cas. Dans l'exemple ci-dessous, pour préparer une tasse de thé, il faut nécessairement faire bouillir de l'eau.

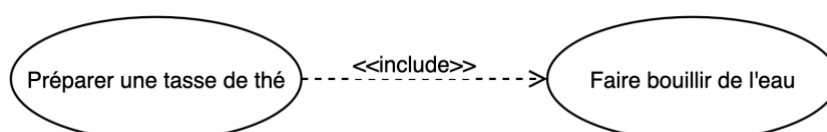


Figure 11 - Représentation UML d'une inclusion d'un cas d'utilisation

Diagramme



Figure 12 - diagramme de cas d'utilisation

Explications :

Un utilisateur non authentifié peut postuler en envoyant sa candidature via le formulaire. Les données sont ensuite enregistrées dans la base de données et les fichiers attachés sont stockés sur le serveur de fichiers.

Un administrateur doit d'abord s'authentifier via le système d'authentification. Une fois connecté, il peut :

- consulter la liste des candidatures
- trier les candidatures par note, nom (ordre alphabétique) ou date de soumission
- consulter les détails d'une candidature
- ajouter un commentaire avec une note
- consulter les fichiers joints à une candidature

4.4.2. Description

Les besoins des utilisateurs proviennent du cahier des charges et des échanges avec le chef de projet. Ils sont répartis entre les utilisateurs (candidats) et les administrateurs (recruteurs/formateurs).

Utilisateurs (non authentifiés)

Les besoins de cette section sont prévus pour tous les utilisateurs non authentifiés.

Envoyer une candidature	
Quoi	Afficher un formulaire permettant de soumettre un dossier de candidature
Pourquoi	Permettre aux candidats de postuler pour une place d'apprentissage
Comment	Formulaire web

Authentification	
Quoi	Rediriger l'utilisateur vers une page de connexion
Pourquoi	Identifier les utilisateurs, protéger les données sensibles des candidats et restreindre l'accès aux fonctionnalités d'édition (ajout, modification, suppression de commentaires et notes)
Contrainte	L'utilisateur doit posséder un identifiant valide
Remarque	L'utilisateur est ensuite redirigé vers la page initialement demandée

Administrateurs (utilisateurs authentifiés)

Les besoins de cette section sont prévu pour tout les utilisateurs authentifiés et administrateurs.

Consulter la liste des candidatures	
Quoi	Afficher la liste de toutes les candidatures
Pourquoi	Visualiser les candidatures, les trier, accéder aux détails
Comment	Interface web

Trier les candidatures	
Quoi	Trier par note, nom (ordre alphabétique) ou date de soumission
Pourquoi	Faciliter l'évaluation des dossiers
Comment	Trois boutons de tri au-dessus de la liste, permettant un tri croissant/décroissant
Remarque	Le tri par date est appliqué par défaut

Consulter les détails d'une candidature	
Quoi	Afficher les informations détaillées d'une candidature
Pourquoi	Permettre une évaluation complète et précise
Comment	Interface web

Ajouter un commentaire	
Quoi	Zone de texte sur la page des détails
Pourquoi	Permettre aux recruteurs de donner leur avis sur chaque candidat et de le partager avec les autres
Comment	Interface web

Modifier un commentaire	
Quoi	Zone de texte sur la page des détails
Pourquoi	Autoriser les recruteurs à mettre à jour leur avis
Comment	Interface web
Remarque	L'utilisateur doit être l'auteur du commentaire

Ajouter une note	
Quoi	Attribuer une note de 1 à 5 étoiles à un dossier
Pourquoi	Quantifier l'évaluation et faciliter le classement
Comment	Sélecteur d'étoiles intégré à la zone de commentaire

Consulter les fichiers d'une candidature	
Quoi	Accéder aux pièces jointes
Pourquoi	Permettre aux recruteurs d'examiner tous les justificatifs fournis
Comment	Lien par fichier dans la page des détails
Remarque	Le lien ouvre le document dans un nouvel onglet (lecture ou téléchargement possible)

Synthèse

Récapitulatif des besoins selon le rôle de l'utilisateur :

Besoins	Utilisateurs non authentifiés	Administrateurs
Authentification	X	X
Envoyer une candidature	X	X
Consulter la liste des candidatures		X
Trier les candidatures		X
Consulter les détails d'une candidature		X
Ajouter un commentaire		X
Modifier un commentaire		X
Supprimer un commentaire		X
Ajouter une note		X
Consulter les fichiers d'une candidature		X

4.5. Fonctionnement

Cette section analyse et compare les solutions disponibles afin de répondre au mieux aux besoins exprimés, tout en tenant compte des différentes contraintes spécifiques au contexte de ce projet.

4.5.1. Système d'authentification

Variantes du système d'authentification

Plusieurs méthodes d'authentification sont envisageables pour ce projet, chacune présentant ses avantages et inconvénients.

Barème des critères de comparaison			
Critère	1 : Bas	2 : Moyen	3 : Haut
Sécurité	Un seul facteur (ex : email/mot de passe)	Deux facteurs ou annuaire central sans MFA	Authentification sans mot de passe ou SSO centralisé (Single sign-on) avec MFA
Expérience utilisateur (UX)	Nouveau compte uniquement pour cette application, flux non-commun	Flux familier	SSO institutionnel, flux connue : friction quasi nulle
Connaissance de la technologie	Technologie jamais utilisée auparavant	Technologie connue, mais peu pratiquée	Technologie maîtrisée et déjà utilisée dans d'autres projets

Matrice de comparaison				
Méthode	Sécurité	UX	Connaissance	Note finale*
Nom d'utilisateur et mot de passe local	1	2	3	1.7
Annuaire LDAP HES-SO//FR	2	2	1	1.8
OTP (One-time password, ex : par SMS / e-mail / application mobile)	2	1	1	1.5
Passkey	3	2	1	2.3
CA TIC Authentication Gateway (Microsoft MFA)	3	3	3	3

Pondération : Sécurité 50%, UX 30%, Connaissance 20%

*Formule : Note = S*0.5 + UX*0.3 + Tech *0.2

CA TIC Authentication Gateway (Microsoft MFA)

La CA TIC Authentication Gateway est une passerelle d'authentification développée par le Centre d'Apprentissage TIC de la HEIA-FR. Elle permet l'authentification via les comptes Microsoft des utilisateurs, en simplifiant l'intégration dans les applications et en évitant d'avoir à enregistrer manuellement une nouvelle application dans Microsoft Entra.

Conclusion

Le choix s'est porté sur l'utilisation de la CA TIC Authentication Gateway pour les raisons suivantes :

1. Expérience utilisateur (UX)

Le flux d'authentification via le compte Microsoft du tenant HES-SO//FR est familier aux utilisateurs. De plus, la majorité des utilisateurs sont déjà connectés à leur compte Microsoft sur leur poste professionnel. Dans ce cas, ils accèdent directement à l'interface protégée sans devoir s'authentifier à nouveau.

2. Sécurité

L'authentification s'appuie sur les mécanismes de sécurité Microsoft, avec support natif de l'authentification multifactor (MFA) selon les règles et paramètres de sécurité définis par le tenant.

3. Connaissance de la technologie

La solution est déjà maîtrisée en interne et a été utilisée avec succès dans plusieurs projets précédents.

Remarque : Les utilisateurs HES-SO//FR devant accéder à l'interface en tant qu'administrateurs doivent être ajoutés manuellement à une base de données locale sur le serveur de la passerelle.

Fonctionnement

- Authentification d'un utilisateur : Redirection de l'utilisateur vers la passerelle d'authentification, la passerelle le redirige sur une page de connexion. Une fois l'utilisateur authentifié avec son compte Microsoft, l'utilisateur est redirigé vers la page initialement demandée.
- Déconnexion d'un utilisateur : Redirection de l'utilisateur vers la page de déconnexion de la passerelle. L'utilisateur est ensuite redirigé vers la page initiale.

4.5.2. Formulaire

Champs du formulaire

Par rapport à la version papier, le formulaire numérique offre une expérience optimisée : validation automatique des données, affichage dynamique des champs conditionnels, et sélecteurs ergonomiques (listes déroulantes, cases à cocher). Chaque champ est associé à un type spécifique et à des contraintes claires.

Champ	Type	Restrictions
Informations personnelles		
Prénom / Nom	Texte	- Obligatoire - Max. 50 caractères ou moins
Date de naissance	Date	- Obligatoire - Format : JJ.MM.AAAA - Antérieure à la date du jour
Téléphone	Texte	- Obligatoire - Max. 25 caractères ou moins - Peut être suisse (+41) ou étranger
Email	Email	- Obligatoire - Format email - Max. 254 caractères ou moins
Rue + n°	Texte	- Obligatoire - Max. 60 caractères ou moins
NPA	Texte	- Obligatoire - Max. 15 caractères ou moins
Localité	Texte	- Obligatoire - Max. 30 caractères ou moins
Pays	Texte	- Obligatoire - Valeur par défaut : « Suisse » - Max. 50 caractères ou moins
Formation		
Actuellement au CO	Booléen	- Obligatoire

Année CO / Type de classe	Radio	- Désactivé si "Actuellement au CO est faux"
Fichiers		
Photo du candidat	Fichier	- Obligatoire - Format PNG, JPG, JPEG
CV, lettre de motivation, bulletins scolaires, attestations de stages, autres	Fichier	- Obligatoire - Format PNG, JPG, JPEG, PDF

Les restrictions des différents types ont été inspirées d'un exemple de base de donnée Microsoft (AdventureWorks) :

https://dataedo.com/samples/html/AdventureWorks/doc/AdventureWorks_2/home.html

4.5.3. Administration

Vue de la liste des candidatures	
Action	Description
Affichage de la liste de toutes les candidatures	Afficher les candidatures sous forme d'une liste d'éléments. Chaque ligne de cette liste contient : • La photo du candidat • Le prénom et le nom • NPA + Ville, pays • Note d'évaluation • Un bouton pour afficher les commentaires dans une fenêtre de type overlay • Un bouton pour accéder à la page des détails de la candidature Si aucune note n'a été attribuée, la note d'évaluation apparaîtra en gris.
Trier la liste des candidatures	Le tri des candidatures est possible selon les critères suivants : • Note d'évaluation • Nom de famille (ordre alphabétique) • Date de réception Par défaut, la liste est triée selon la date de réception afin que les candidatures les plus récentes soient visibles en haut de la liste et traitées en priorité par les administrateurs.

Affichage rapide des commentaires	Le bouton commentaire présent sur chaque ligne de la liste permet d'afficher une fenêtre overlay (fenêtre apparaissant au-dessus du contenu). La fenêtre overlay affiche tous les commentaires associés à la candidature. Pour chaque commentaire, sont indiqués : • L'auteur (Prénom + Nom de l'administrateur) • Le texte du commentaire avec prise en compte des retours à la ligne • La note d'évaluation associée au commentaire • Un bouton permet de fermer cette fenêtre overlay. Si aucun commentaire n'a été ajouté à la candidature, un message informatif sera affiché dans la fenêtre.
-----------------------------------	---

Vue des détails d'une candidature	
Action	Description
Affichage des détails de la candidature	Une première section contient les informations personnelles du candidat : • Prénom + Nom • Adresse email • Numéro de téléphone • Âge calculé à partir de la date de naissance • Photo du candidat La note moyenne du candidat est affichée au-dessus de son prénom et nom. La date de réception de la candidature est affichée en haut à droite du formulaire. Une deuxième section contient les informations relatives à la formation du candidat : • Si le candidat est actuellement au CO • Le type de classe • L'année de CO Si le candidat n'est pas actuellement au CO, le type de classe et l'année de CO ne sont pas affichés. Une troisième section contient l'adresse du candidat : • Adresse complète • NPA • Pays

Consulter un document	Une section « Documents » affiche la liste des documents attachés à la candidature. Chaque document est présenté sous forme de lien cliquable permettant de le consulter.
Ajouter un commentaire et une note	Une section « Commentaires » contient la liste des commentaires : • Si l'administrateur n'a pas encore ajouté de commentaire, un espace d'édition est disponible. • Si l'administrateur a déjà ajouté un commentaire, deux boutons apparaissent : « Supprimer » et « Modifier ».

Variantes pour consulter un document

Il existe plusieurs façons pour un recruteur ou un formateur de consulter un document attaché à une candidature, tel qu'un CV, une lettre de motivation, des bulletins de notes, ou des attestations de stages.

Contexte :

- Une candidature contient en moyenne 5 documents.
- Les recruteurs reçoivent plus de 70 candidatures par année.
- Chaque recruteur devra donc consulter environ 350 documents au total.

Variante	Description
Téléchargement direct (lien déclenchant le téléchargement)	Avantages : - Implémentation très simple - Lecture possible hors-ligne Inconvénients : - Mauvaise expérience utilisateur (3 étapes : télécharger, ouvrir l'explorateur, double-cliquer) - Nettoyage manuel requis des fichiers locaux
Aperçu dans un nouvel onglet (ouverture du PDF dans la visionneuse native du navigateur)	Avantages : - Bonne expérience utilisateur (1 clic, rendu PDF par le navigateur) - Possibilité de télécharger/imprimer depuis l'onglet Inconvénients : - Multiplication des onglets lors d'une revue prolongée - Absence de navigation directe entre les pièces jointes (suivant/précédent) - Perte temporaire du contexte (changement d'onglet)

Aperçu intégré dans la page (iframe/PDF.js)	Avantages : - Navigation fluide sans changement de page ou d'onglet - Ajout possible de fonctionnalités (zoom, miniature des pages, annotation) Inconvénients : - Implémentation plus complexe - Charge initiale de la page plus lourde, surtout sur mobile - Réduction potentielle de l'espace disponible pour les autres informations du candidat
Aperçu dans une fenêtre modale (lightbox pleine largeur/hauteur)	Avantages : - Expérience utilisateur rapide (1 clic, fermeture facile avec ESC) - Conserve le contexte de la candidature (pas d'onglet supplémentaire) - Possibilité d'intégrer un carrousel (navigation Suivant / Précédent) Inconvénients : - Implémentation plus complexe - Sur petits écrans, espace réduit pour les documents A4, nécessitant du zoom/scroll
Téléchargement groupé (ZIP)	Avantages : - 1 clic pour récupérer tous les documents, pratique pour lecture hors-ligne ou archivage - Réduit le nombre de requêtes réseau au serveur Inconvénients : - Deux étapes supplémentaires nécessaires pour chaque fichier après extraction - Temps de téléchargement potentiellement long (> 10 Mo)

Conclusion

Choix final : Aperçu dans un nouvel onglet (ouverture du PDF dans la visionneuse native du navigateur)

Après avoir présenté ces variantes au chef de projet, la décision a été prise d'implémenter l'aperçu des documents dans un nouvel onglet, déclenché lors du clic sur le lien du fichier. Cette solution permet :

- De consulter rapidement le document en un seul clic
- De conserver la possibilité de télécharger ou d'imprimer le document facilement

4.6. Ébauches conceptuelles

Authentification

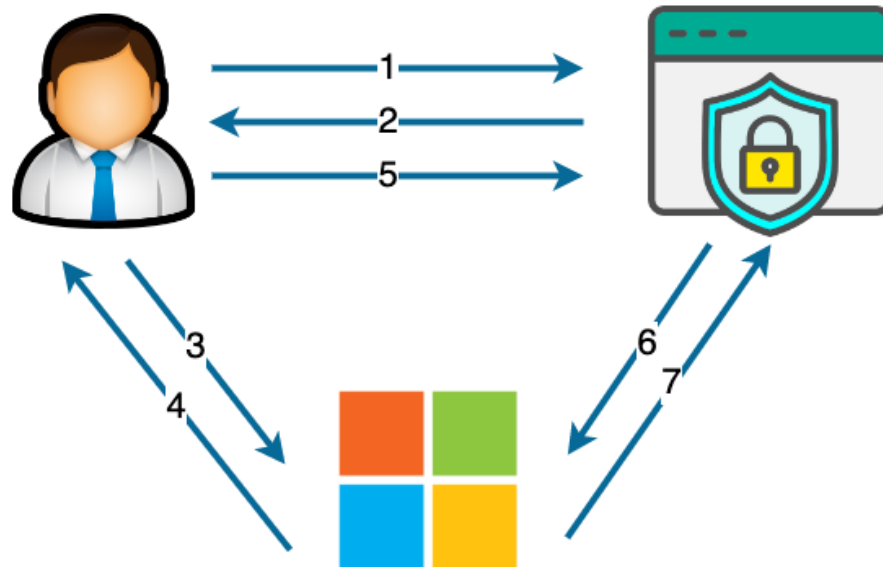


Figure 13 - Ébauche conceptuelle: Authentification

Description :

1. L'utilisateur non authentifié accède à une page protégée de l'application.
2. L'application redirige l'utilisateur vers la « CA TIC Authentication Gateway ».
3. L'utilisateur se connecte à la « CA TIC Authentication Gateway » avec son identifiant Microsoft.
4. La « CA TIC Authentication Gateway » redirige l'utilisateur vers la page protégée initialement demandée, et celui-ci obtient un jeton.
5. L'utilisateur accède à la page de l'application avec son jeton.
6. L'application vérifie le jeton auprès de la « CA TIC Authentication Gateway ».
7. La « CA TIC Authentication Gateway » indique à l'application la validité du jeton.

Cas possibles :

- Si le jeton est valide et que l'utilisateur est enregistré comme admin de eRecrutement dans la passerelle d'authentification, il accède à la page.
- Si le jeton n'est pas valide, l'utilisateur doit recommencer le processus.
- Si le jeton est valide, mais que l'utilisateur n'a pas les droits nécessaires, il est redirigé vers une page d'erreur le lui indiquant.

Envoi d'une candidature

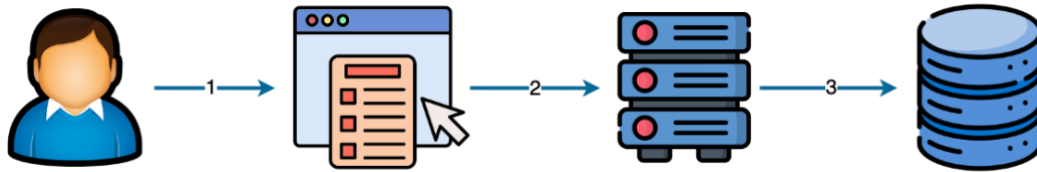


Figure 14 - Ébauche conceptuelle: Envoi d'une candidature

Description :

1. L'utilisateur remplit le formulaire et clique sur le bouton pour l'envoyer.
2. Le formulaire valide les données et les transmet au serveur web.
3. Les données sont vérifiées et traitées ; les fichiers sont enregistrés sur le serveur, et les informations sont stockées dans la base de données.
4. L'utilisateur est redirigé vers une page de confirmation.

Afficher la liste des candidatures



Figure 15 - Ébauche conceptuelle: Affichage de la liste des candidatures

Description :

1. L'administrateur accède à l'application.
2. L'application récupère la liste des candidatures depuis la base de données.
3. Les candidatures sont affichées.

Afficher les détails d'une candidature



Figure 16 - Ébauche conceptuelle: Affichage des détails d'une candidature

Description :

1. L'administrateur accède à la page des détails d'une candidature.
2. L'application récupère les détails de la candidature depuis la base de données.
3. Les détails sont affichés.

Ajouter, modifier, supprimer un commentaire et note

Un commentaire peut être ajouté sans note, et une note peut être ajoutée sans commentaire.

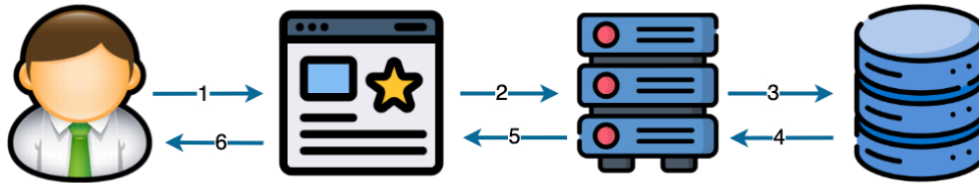


Figure 17 - Ébauche conceptuelle: Ajouter, modifier, supprimer un commentaire et note

Ajouter un commentaire et une note

Un administrateur peut ajouter un commentaire et une note seulement s'il n'y a pas déjà un commentaire de sa part sur la même candidature.

Description :

1. L'administrateur ajoute un commentaire et une note dans l'espace dédié sur la page des détails.
2. L'application envoie ces données au serveur web.
3. Le serveur vérifie l'absence d'un commentaire du même auteur sur la même candidature, puis ajoute le commentaire et la note à la base de données.
4. La base de données confirme l'ajout.
5. La confirmation est transmise à l'application.
6. Le commentaire et la note sont visibles par l'administrateur, et la note moyenne est mise à jour.

Modifier un commentaire et une note

Un administrateur ne peut que modifier son propre commentaire.

Description :

1. L'administrateur clique sur le bouton d'édition, modifie le texte et la note, puis confirme.
2. L'application envoie les modifications au serveur.
3. Le serveur met à jour les données dans la base.
4. La base confirme la modification.
5. La confirmation est transmise à l'application.
6. L'administrateur voit le commentaire mis à jour ; la note moyenne est recalculée.

Supprimer un commentaire et une note

Un administrateur ne peut que supprimer son propre commentaire.

Description :

1. L'administrateur clique sur le bouton de suppression.
2. L'application transmet la requête au serveur.
3. Le serveur supprime le commentaire de la base de données.
4. La base confirme la suppression.
5. La confirmation est transmise à l'application.
6. Le commentaire disparaît et la note moyenne est mise à jour.

4.6.1. Consulter les fichiers d'une candidature

L'administrateur peut consulter les fichiers d'une candidature en cliquant sur le lien correspondant sur la page des détails. Un nouvel onglet contenant le document s'ouvre alors automatiquement dans le navigateur.

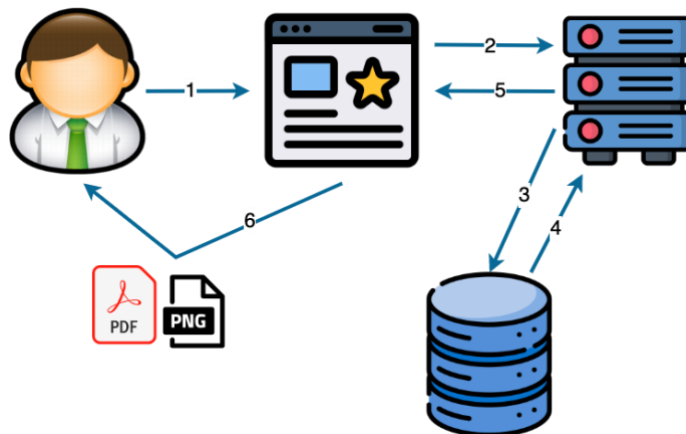


Figure 18 - Ébauche conceptuelle: Consulter les fichiers d'une candidature

Description :

1. L'administrateur clique sur le lien du fichier qu'il souhaite consulter depuis la page des détails d'une candidature.
2. L'application envoie une requête au serveur web pour récupérer le fichier demandé.
3. Le serveur web interroge la base de données afin d'obtenir l'emplacement du fichier.
4. La base de données renvoie l'emplacement du fichier stocké sur le serveur web.
5. Le serveur web récupère le fichier et le transmet à l'application en réponse à la requête initiale.
6. L'application ouvre automatiquement un nouvel onglet pour afficher le fichier image ou document. Si le fichier est au format PDF, il s'affichera dans le lecteur PDF natif du navigateur.

4.7. Sécurité et risques

Le formulaire de candidature est accessible sans authentification par tout le monde sur Internet. Il représente donc une porte d'entrée directe vers l'application : n'importe quel internaute ou robot peut y soumettre des données.

Plusieurs risques de sécurité doivent être pris en compte, et diverses mesures doivent être appliquées lors du développement de l'application afin de minimiser ces risques.

4.7.1. Upload des fichiers

Le formulaire contient des champs pour les données du candidat, de type texte, booléen, date, email, numérique, ainsi que des fichiers pour la photo du candidat et les pièces jointes à son dossier de candidature, telles que :

- CV, lettre de motivation : Documents PDF
- Bulletins de notes, attestations de stages, photo du candidat : Images PNG, JPG ou JPEG

Plusieurs risques existent lorsqu'un formulaire en ligne permet de téléverser des fichiers :

Attaque de l'infrastructure

Exemples :

- Si un fichier portant le même nom et la même extension qu'un fichier existant sur le serveur est téléchargé, il pourrait écraser le fichier existant.
- Si un fichier contient un programme malveillant ou un logiciel d'exploitation d'une vulnérabilité dans le traitement des fichiers côté serveur, il pourrait être utilisé pour prendre le contrôle du serveur, entraînant de graves conséquences.

Attaque contre les utilisateurs

Exemple : Si un fichier uploadé contient un logiciel malveillant, un script ou une macro malveillante, il pourrait permettre à un attaquant de prendre le contrôle des machines des utilisateurs infectés lorsque ceux-ci ouvrent ou téléchargent le fichier.

Attaque par déni de service

Exemple : L'upload d'un fichier extrêmement volumineux peut entraîner une forte consommation des ressources du serveur et perturber la disponibilité du service.

Sources pour davantage d'exemples et de détails sur les types d'attaques et risques :

- https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- <https://www.opswat.com/blog/file-upload-protection-best-practices>

Solutions

Pour réduire les risques liés à l'envoi et au stockage des fichiers provenant d'Internet, voici les bonnes pratiques recommandées par les sources citées ci-dessus. Les mesures ci-après sont prioritaires dans le contexte du projet. Bien qu'il ne soit pas possible d'implémenter toutes les solutions possibles, ces mesures contribuent significativement à réduire les possibilités d'attaques.

Mesure	Description et choix
Autoriser uniquement certains types de fichiers	Les formats PDF, PNG, JPG et JPEG couvrent les besoins du projet (CV, lettre de motivation, photo du candidat, bulletins de notes, attestations, certificats).
Limiter la longueur des noms de fichiers	Les noms de fichiers de plus de 255 caractères ne seront pas acceptés. 255 caractères constituent la limite sur macOS, Windows et Linux.
Limiter le volume total des fichiers uploadés	- Maximum 25 Mo pour l'ensemble des fichiers attachés - Maximum 10 Mo pour un fichier attaché.
Utiliser des messages d'erreur simple	En cas d'erreur lors de la sauvegarde des fichiers, aucune donnée sensible (par exemple, le chemin du répertoire) ne sera divulguée.
Renommer les fichiers avant leur enregistrement	Les fichiers seront renommés avec un UUID (Universally Unique Identifier) avant d'être enregistrés.
Enregistrer les fichiers dans un répertoire non accessible depuis le web	Les fichiers seront stockés dans un répertoire inaccessible publiquement depuis le web. La route d'accès aux fichiers sera protégée par authentification.
Désamorcer le contenu via CDR (Content Disarm & Reconstruct)	CDR (Content Disarm & Reconstruct) est une technique de sécurité qui ne cherche pas à détecter la présence de code malveillant dans un fichier : elle le neutralise par principe. 1. Le fichier uploadé est ouvert dans une sandbox isolée. 2. Tous les éléments actifs ou non standard (macros, scripts, métadonnées, etc.) sont supprimés. 3. Le contenu « innocent » (texte, images, mise en page) est reconstruit dans un nouveau fichier propre et ré-encodé au format attendu. Le résultat est visuellement identique pour l'utilisateur, mais garantit qu'aucun code exécutable ou donnée cachée ne subsiste, éliminant ainsi toute attaque liée au fichier original.

4.7.2. Envoi du formulaire

Voici les trois principaux risques identifiés liés aux champs du formulaire si les standards de sécurité ne sont pas correctement implémentés :

- Injection SQL
- Cross-Site Scripting (XSS) stocké
- Soumissions automatisées (spam du formulaire)

Validation et assainissement des entrées

Une défense robuste contre l'injection SQL, le XSS ou les soumissions frauduleuses commence par un contrôle strict des données dès le navigateur, puis une vérification supplémentaire côté serveur. Les règles appliquées s'appuient sur les contraintes déjà définies pour chaque champ lors de l'analyse des besoins (type, longueur, format, etc.).

Pourquoi valider deux fois ?

- Frontend : offre un retour immédiat à l'utilisateur (UX), limite les erreurs involontaires.
- Backend : reste indispensable, car les validations côté client peuvent être désactivées ou contournées par un attaquant. Le serveur est l'ultime gardien de l'intégrité des données.

Protection anti-spam

Sans défenses spécifiques, le formulaire public peut être ciblé par des robots, entraînant les conséquences suivantes :

- Une inondation de candidatures factices rendant l'application inutilisable pour les administrateurs.
- Un déni de service : l'envoi massif de formulaires consomme toutes les ressources du serveur, rendant l'application indisponible.

Plusieurs solutions peuvent être mises en place pour réduire le risque de spam, notamment l'utilisation d'un système de CAPTCHA. Cependant, le contexte actuel du projet ne permet pas la mise en œuvre immédiate de ces mesures de sécurité. Celles-ci feront toutefois l'objet d'améliorations potentielles lors des futures évolutions de l'application.

Protocole HTTPS

Durant le développement, le protocole HTTP est utilisé pour simplifier les tests. L'application est alors développée et testée localement, avec toutes les données et l'application résidant sur le même ordinateur ou au sein d'un réseau local. Cependant, une fois en production, il est **impératif d'utiliser le protocole HTTPS** pour garantir la sécurité des données échangées entre les utilisateurs et le serveur.

4.8. Technologie utilisées

Technologie	Description
Express	Framework web minimaliste et flexible pour Node.js ; il fournit un système de routage robuste, un vaste écosystème de middlewares et simplifie la création d'API et d'applications web. Source : https://expressjs.com
MySQL	Système de gestion de base de données relationnelle (SGBDR) open-source à large diffusion, reconnu pour ses performances, sa fiabilité et la richesse de son écosystème SQL. Source : https://dev.mysql.com/doc
Node.js	Environnement d'exécution JavaScript côté serveur, bâti sur le moteur V8 de Chrome ; il offre un modèle événementiel non bloquant qui permet de créer des applications réseau performantes et scalables. Source : https://nodejs.org
REST / RESTful	Style architectural pour la conception d'API : il repose sur les verbes HTTP (GET, POST, PUT, DELETE...), l'utilisation d'URI pour représenter les ressources, l'absence d'état côté serveur (stateless) et des formats d'échange courants (JSON, XML), favorisant l'interopérabilité et la simplicité. Source : https://restfulapi.net
Vue	Vue est un framework JavaScript pour créer des interfaces utilisateur. Il se base sur les standards HTML, CSS et JavaScript et offre un modèle de programmation déclaratif et basé sur des composants, facilitant le développement efficace d'interfaces utilisateur de toute complexité. Source: https://vuejs.org

5. Conception

5.1. Schéma d'architecture

L'application repose sur une architecture en trois tiers, où chaque tiers remplit un rôle spécifique :

- **Présentation (navigateur)** : Gère la présentation des données et l'interaction avec l'utilisateur.
- **Traitement (serveur web)** : Réalise les traitements métiers, appliquant l'ensemble des règles de gestion ainsi que la logique applicative.
- **Données (base de données)** : Assure le stockage sécurisé et l'accès efficace aux données.

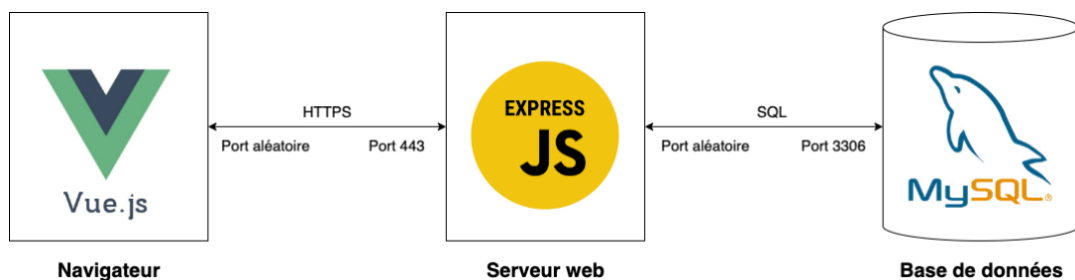


Figure 19 - Schéma d'architecture 3-tiers

Source : https://fr.wikipedia.org/wiki/Architecture_trois_tiers

5.2. Diagrammes de séquences

Authentification

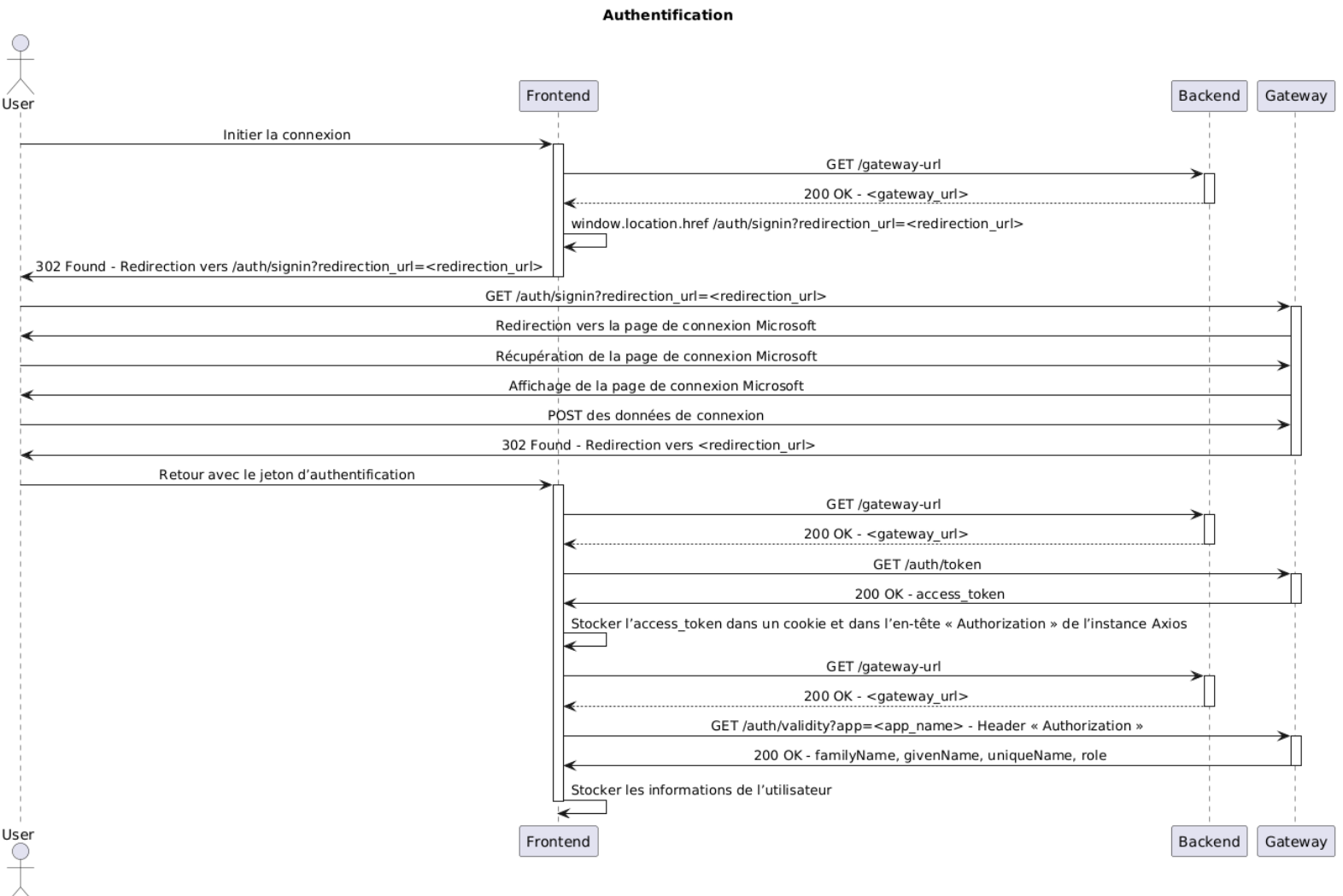


Figure 20 - Diagramme de séquence: Authentification

L'utilisateur déclenche le flux d'authentification depuis l'interface Frontend, qui émet une requête HTTP GET sur /gateway-url vers le Backend afin de récupérer l'URL de la passerelle d'authentification. Le Backend retourne une réponse 200 OK contenant cette URL. Le Frontend initie alors une redirection HTTP 302 vers la Gateway, laquelle oriente l'utilisateur vers la page de login Microsoft. Après soumission des identifiants, la Gateway émet un jeton d'accès (access_token).

Le Frontend récupère cet access_token via une requête HTTP GET sur /auth/token, puis enregistre le jeton dans un cookie sécurisé et dans l'en-tête Authorization: Bearer <access_token> de l'instance Axios. Enfin, le Frontend interroge l'endpoint /auth/validity?app=<app_name> pour valider le JWT, et stocke localement les claims utilisateur (familyName, givenName, uniqueName, role).

Scénarios d'incident potentiels

- Backend indisponible
- Gateway d'authentification indisponible
- Échec de la récupération de l'access_token
- Rejet de l'access_token lors de la validation

Envoi d'une candidature

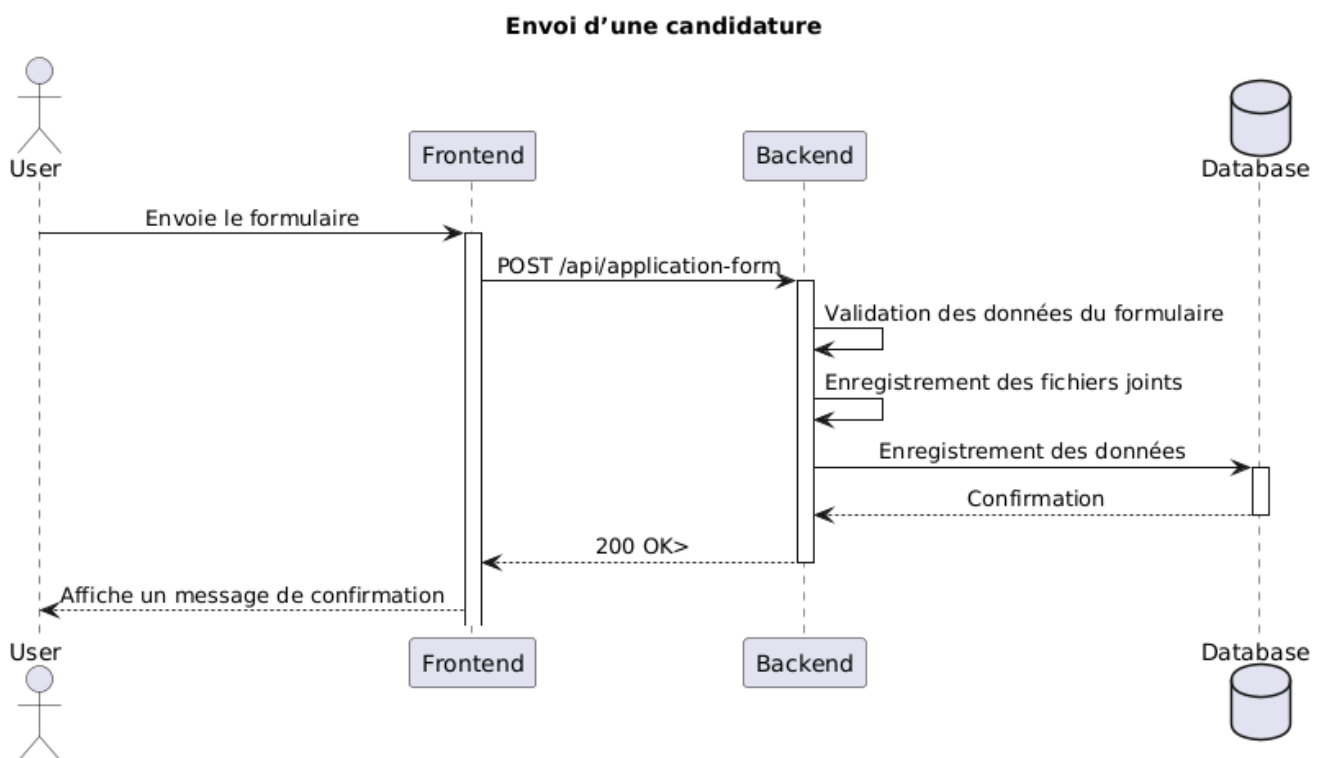


Figure 21 - Diagramme de séquence: Envoi d'une candidature

L'utilisateur remplit le formulaire dans l'interface du Frontend qui envoie les données et fichiers du formulaire avec requête POST au Backend sur /api/application-form, le Backend valide les données du formulaire, si les données et fichiers sont valides, le Backend enregistre les fichiers dans un répertoire en les renommant, puis il enregistre les données dans la base de données, quand toutes les transactions sont confirmées, le Frontend affiche un message de confirmation à l'utilisateur.

Affichage de la liste des candidatures

Affichage de la liste des candidatures

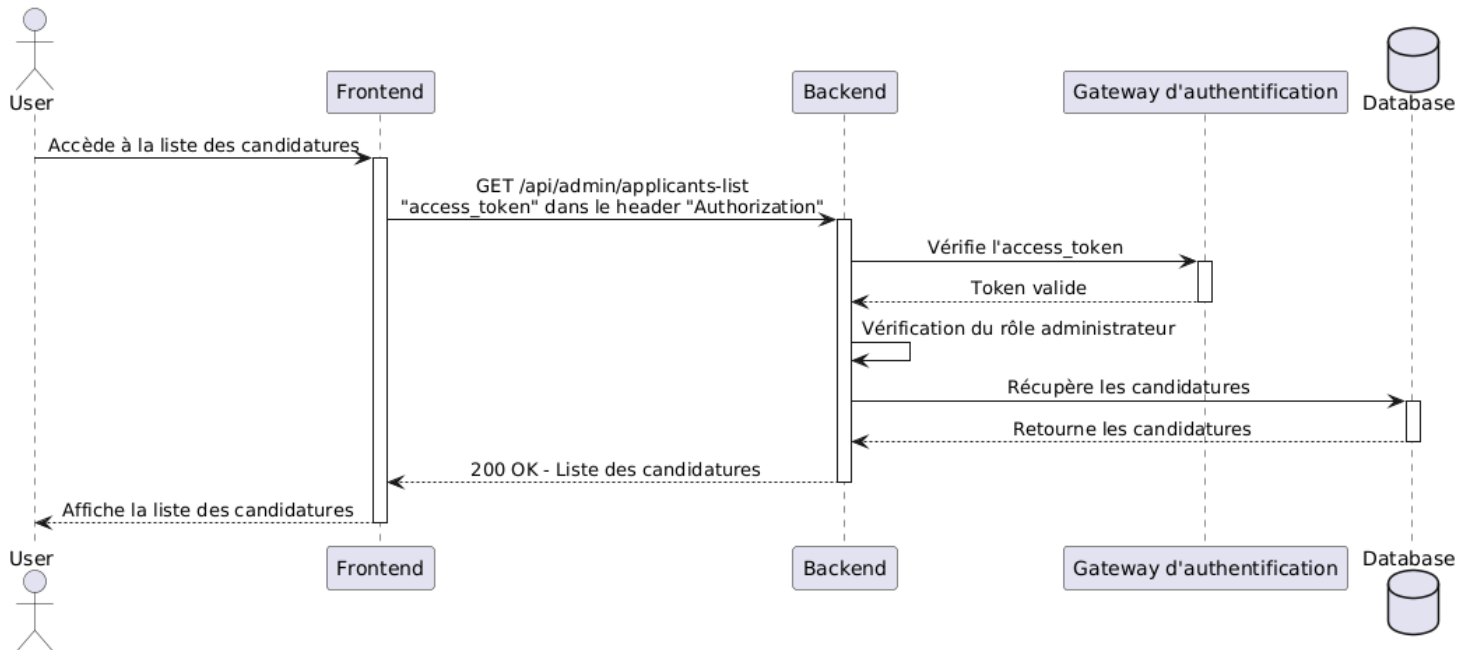


Figure 22 - Diagramme de séquence: Affichage de la liste des candidatures

L'utilisateur ouvre la page de la liste des candidatures ; le Frontend émet alors une requête GET /api/admin/applicants-list contenant l'access_token dans le header Authorization vers le Backend. Le Backend délègue la vérification du jeton à la gateway d'authentification ; lorsque le token est déclaré valide, il contrôle que l'utilisateur possède bien le rôle administrateur. Une fois ces autorisations confirmées, le Backend interroge la base de données pour récupérer toutes les candidatures enregistrées. Après réception des données, il renvoie une réponse 200 OK avec la liste au Frontend, qui l'affiche aussitôt à l'utilisateur.

Affichage des détails d'une candidature

Affichage des détails d'une candidature

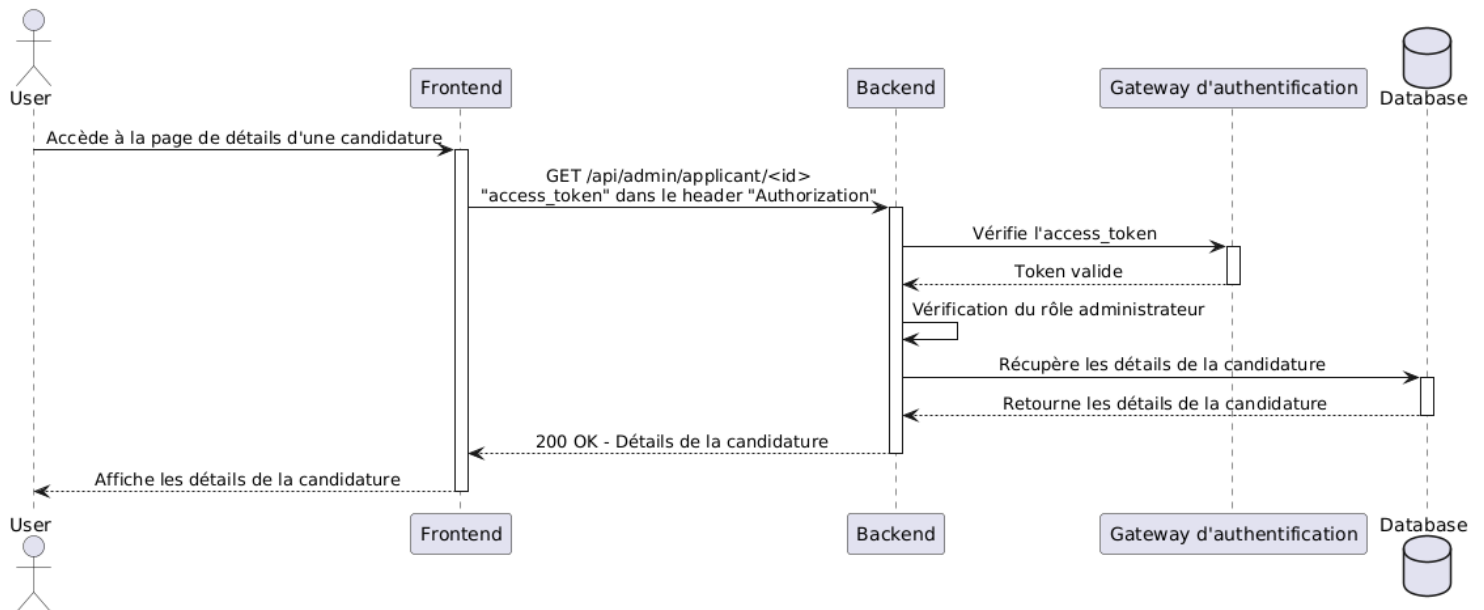


Figure 23 - Diagramme de séquence: Affichage des détails d'une candidature

L'utilisateur ouvre la page de détails d'une candidature ; le Frontend envoie alors au Backend une requête `GET /api/admin/applicant/<id>` avec l'`access_token` dans le header `Authorization`. Le Backend transmet d'abord le jeton au Gateway d'authentification ; quand celui-ci confirme sa validité, le Backend vérifie que l'utilisateur dispose bien du rôle administrateur. Une fois l'autorisation acquise, il interroge la base de données pour obtenir toutes les informations relatives à la candidature demandée. Après réception de ces données, le Backend répond au Frontend avec un `200 OK` contenant les détails de la candidature, que le Frontend présente immédiatement à l'utilisateur.

Affichage des commentaires d'une candidatures

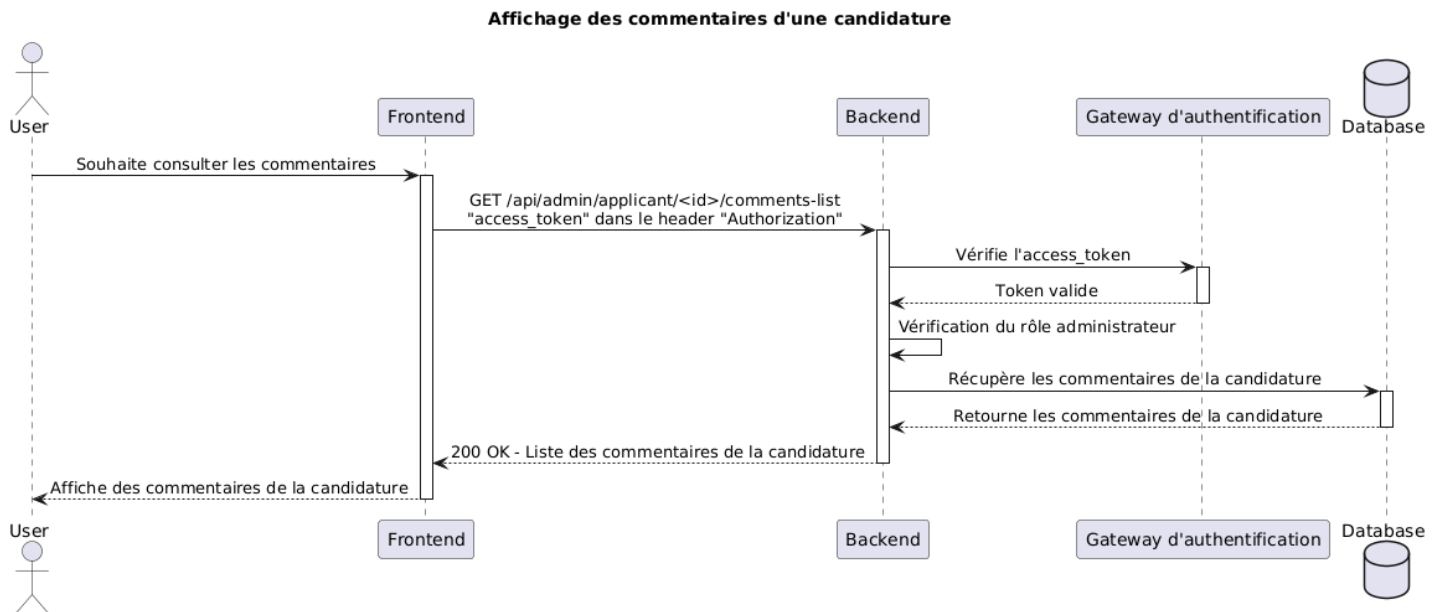


Figure 24 - Diagramme de séquence: Affichage des commentaires d'une candidatures

L'utilisateur appuis sur le bouton pour afficher une fenêtre pour afficher la liste des commentaires d'une candidature ; le Frontend envoie alors au Backend une requête GET /api/admin/applicant/<id>/comments-list avec l'access_token dans le header Authorization. Le Backend transmet d'abord le jeton au Gateway d'authentification ; quand celui-ci confirme sa validité, le Backend vérifie que l'utilisateur dispose bien du rôle administrateur. Une fois l'autorisation acquise, il interroge la base de données pour obtenir les commentaires de la candidature demandée. Après réception de ces données, le Backend répond au Frontend avec un 200 OK contenant les commentaires de la candidature, que le Frontend présente dans une fenêtre overlay à l'utilisateur.

Ajout d'un commentaire

Ajout d'un commentaire

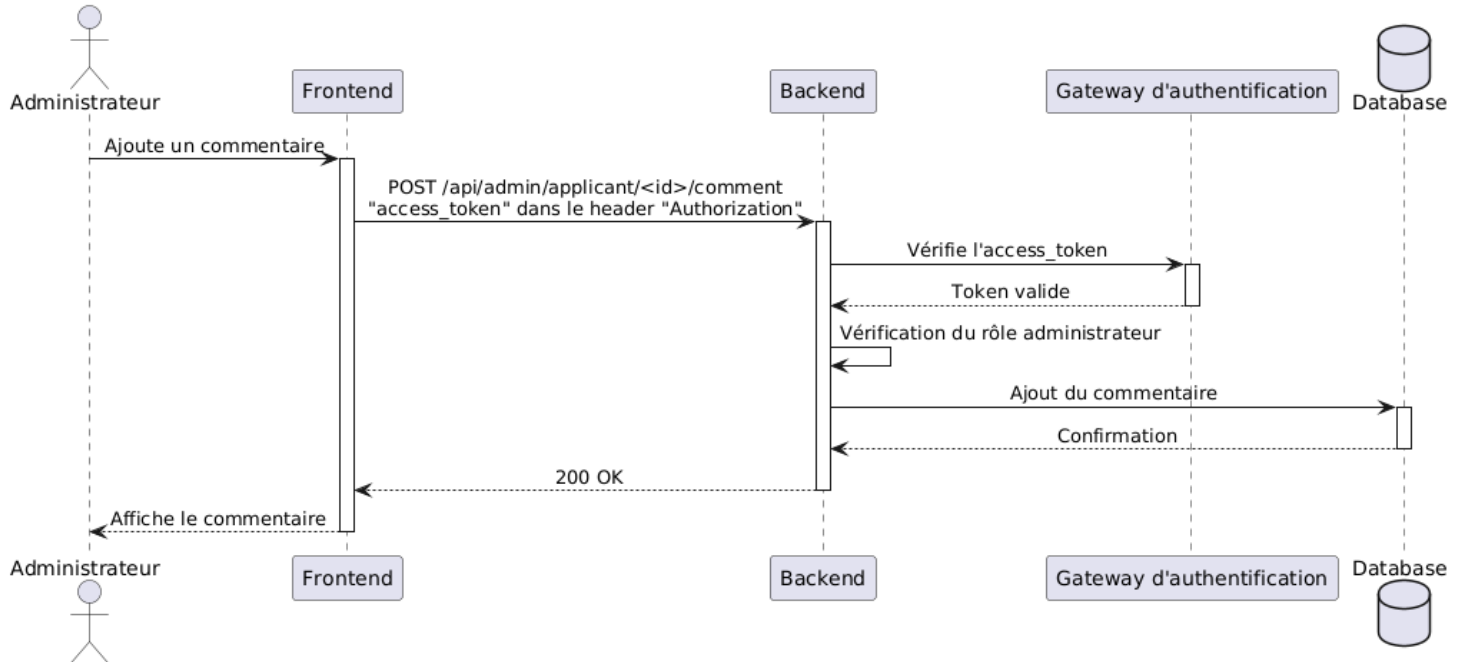


Figure 25 - Diagramme de séquence: Ajout d'un commentaire

L'administrateur saisit son commentaire dans l'interface du Frontend, qui envoie ensuite une requête `POST /api/admin/applicant/<id>/comment` avec le `access_token` dans l'en-tête `Authorization` vers le Backend. Le Backend délègue la vérification du jeton au Gateway d'authentification ; une fois le token validé, il s'assure que l'utilisateur possède bien le rôle administrateur, puis insère le commentaire en base de données. Quand la base confirme l'opération, le Backend renvoie un `200 OK` au Frontend, qui affiche immédiatement le commentaire ajouté à l'administrateur.

Modification d'un commentaire

Modification d'un commentaire

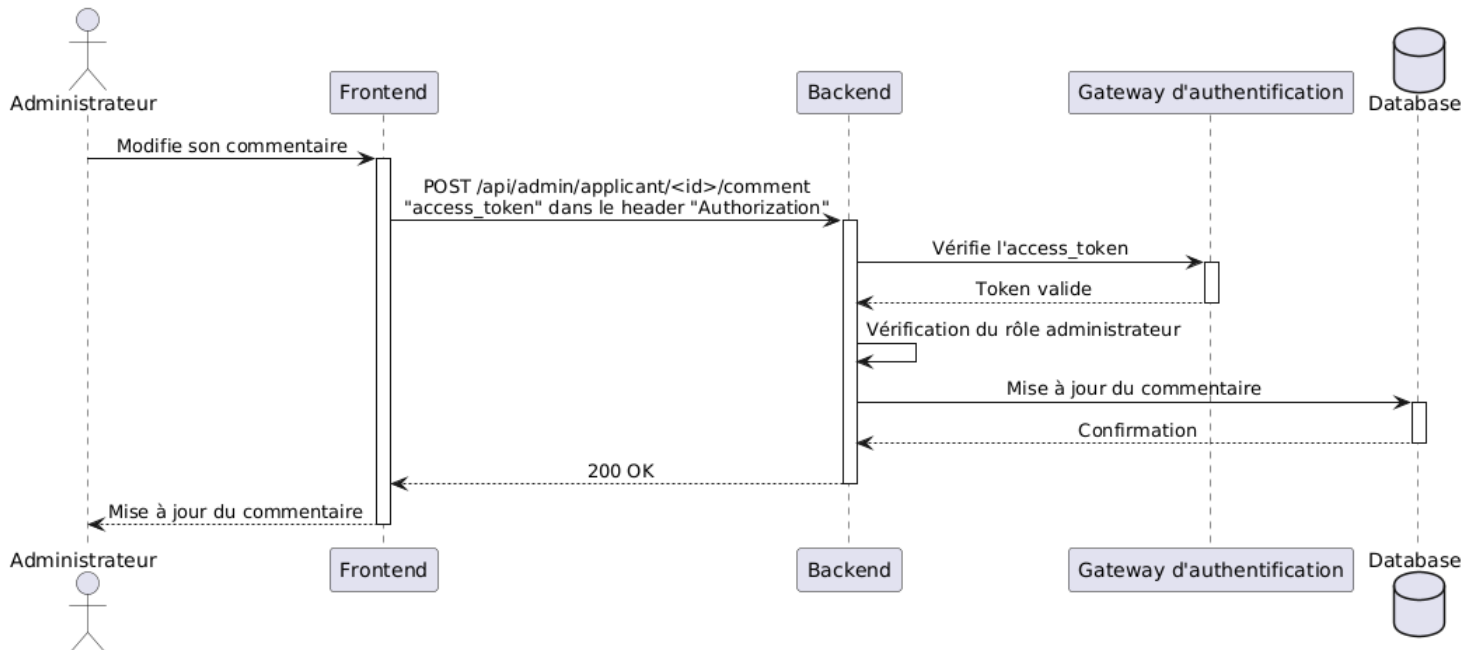


Figure 26 - Diagramme de séquence: Modification d'un commentaire

L'administrateur modifie son commentaire dans l'interface du Frontend, qui envoie ensuite une requête `POST /api/admin/applicant/<id>/comment` avec l'`access_token` dans l'en-tête `Authorization` vers le Backend. Le Backend délègue la vérification du jeton au Gateway d'authentification ; une fois le token validé, il s'assure que l'utilisateur dispose bien du rôle administrateur, puis effectue la mise à jour du commentaire en base de données. Quand la base confirme l'opération, le Backend renvoie un `200 OK` au Frontend, qui actualise l'affichage du commentaire modifié pour l'administrateur.

Suppression d'un commentaire

Suppression d'un commentaire

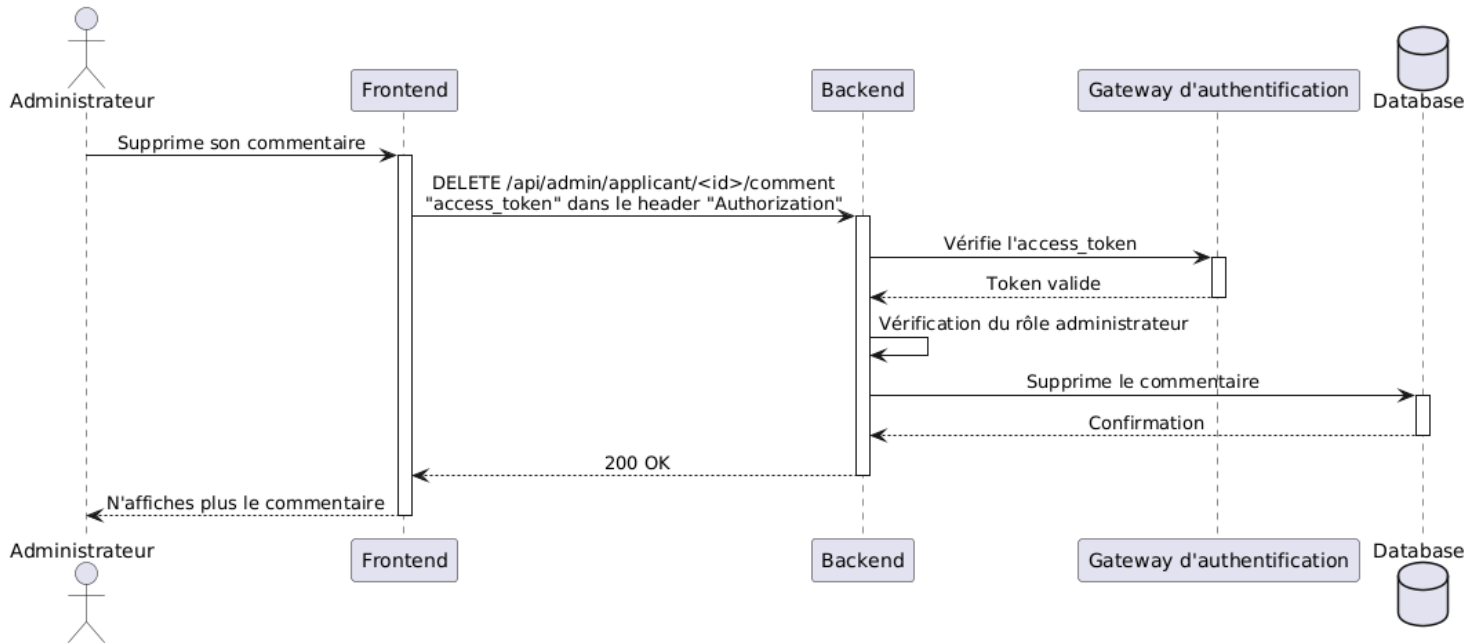


Figure 27 - Diagramme de séquence: Suppression d'un commentaire

L'administrateur supprime son commentaire depuis l'interface du Frontend, qui envoie une requête DELETE /api/admin/applicant/<id>/comment avec le access_token dans l'en-tête Authorization vers le Backend. Le Backend délègue la vérification du jeton au Gateway d'authentification ; une fois validé, il s'assure que l'utilisateur dispose bien du rôle administrateur, puis supprime le commentaire en base de données. Après confirmation de la suppression, il renvoie un 200 OK au Frontend, qui met à jour l'affichage pour ne plus présenter le commentaire à l'administrateur.

Consulter les fichiers d'une candidature

Consulter un fichier d'une candidature

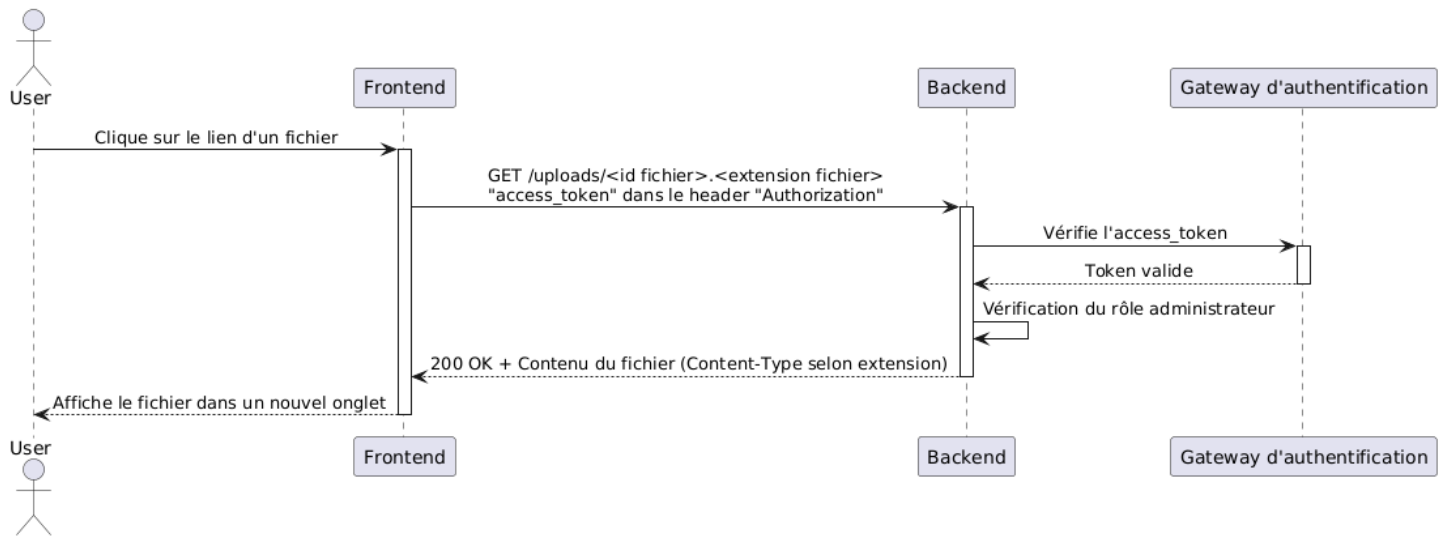


Figure 28 - Diagramme de séquence: Consulter le fichier d'une candidature

L'administrateur clique sur le lien d'un fichier joint dans l'interface du Frontend ; celui-ci envoie alors au Backend une requête GET /uploads/<id fichier>.<extension fichier> avec l'access_token placé dans l'en-tête Authorization. Le Backend transmet d'abord le jeton au Gateway d'authentification ; lorsque celui-ci confirme sa validité, le Backend vérifie que l'utilisateur possède bien le rôle administrateur. Une fois l'autorisation accordée, il récupère le contenu binaire du fichier et répond au Frontend avec un 200 OK incluant le fichier demandé, accompagné du Content-Type approprié selon son extension (PDF, PNG, JPEG, etc.). À réception, le Frontend ouvre le fichier dans un nouvel onglet du navigateur pour le présenter immédiatement à l'administrateur.

5.3. Base de données

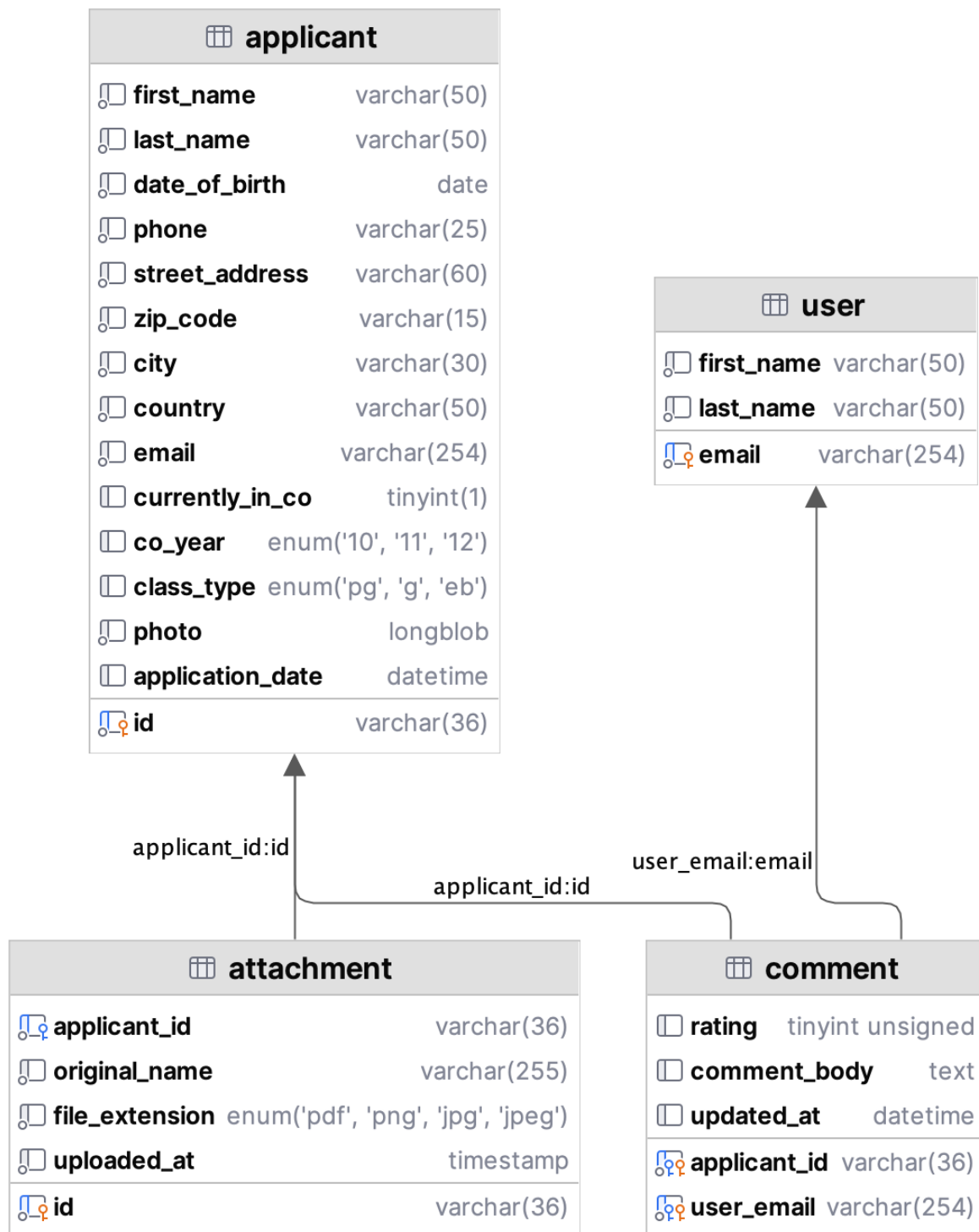


Figure 29 – Schéma de la base de données

Tables

La base de données e_recrutement se compose de quatre tables :

Table	Rôle
applicant	Stocke tous les dossiers de candidature reçus.
user	Contient les administrateurs autorisés à consulter et commenter les dossiers de candidature.
comment	Conserve les notes et remarques déposées par les évaluateurs.
attachment	Garde les fichiers joints (CV, certificats, etc.) liés aux dossiers.

Applicant

Cette table contient les informations des candidatures envoyées. Chaque enregistrement utilise un identifiant unique généré par l'application et utilisé comme clé primaire.

Identifiant : un UUID v4 (colonne id).

Attributs :

- first_name : Prénom du candidat
- last_name : Nom de famille
- date_of_birth : Date de naissance
- phone : Numéro de téléphone
- street_address : Rue et numéro
- zip_code : Code postal
- city : Localité
- country : Pays de résidence
- email : Adresse e-mail de contact
- currently_in_co : Indique si l'élève est toujours au cycle d'orientation (booléen)
- co_year : Année actuelle au CO (10, 11 ou 12)
- class_type : Type de classe (PG, G ou EB)
- photo : Photo d'identité binaire
- application_date : Date et temps de dépôt du dossier

User

Cette table contient la liste des administrateurs autorisés à consulter et commenter les dossiers de candidature.

Identifiant : leur adresse e-mail (colonne email) qui correspond à la même adresse email que celle de leur compte Microsoft HES-SO//FR.

Attributs:

- first_name : Prénom
- last_name : Nom de famille

L'adresse email correspond à celle du compte Microsoft HES-SO//FR de l'administrateur.

Comment

Cette table conserve les notes et remarques déposées par les administrateurs.

Identifiant : clé primaire composite (applicant_id, user_email), car un administrateur ne rédige seulement un commentaire par candidature.

Attributs :

- rating : Note entière (1-5)
- comment_body : Texte du commentaire
- updated_at : Date-heure de la dernière modification (mise à jour automatique)

Attachment

Cette table contient les informations des pièces jointes associées aux dossiers de candidatures.

Identifiant : UUID v4 (colonne id).

Attributs :

- applicant_id : Référence au dossier concerné
- original_name : Nom d'origine du fichier
- file_extension : Extension du fichier (pdf, png, jpg, jpeg)
- uploaded_at : Date et temps de l'upload

Le chemin du fichier peut ensuite être retrouvé :

<Répertoires des fichiers>/<Id du fichier>.<Extension du fichier>

Le nom d'origine du fichier permet de l'afficher à l'administrateur pour qu'il sache de quoi il s'agit (par exemple : cv.pdf) tout en renommant le fichier pour des questions de sécurité définies dans le chapitre précédent.

Cardinalités

Les cardinalités décrivent la nature des relations entre les différentes tables de la base de données, en précisant combien d'enregistrements dans une table peuvent être associés à un enregistrement dans une autre table.

Relation	Type	Description
applicant.id attachment.id	1 :N	Une candidature peut avoir plusieurs fichiers joints, mais un fichiers joint ne peut être associé qu'à une seule candidature.
applicant.id comment.applicant_id	1 :N	Un dossier peut recevoir plusieurs commentaires, mais chaque commentaire vise un seul dossier.
user.email comment.user_email	1 :N	Un évaluateur peut commenter plusieurs dossiers ; la clé composite bloque le doublon sur un même dossier.

Restrictions sur les contraintes d'intégrité référentielle

L'intégrité référentielle joue un rôle fondamental dans la fiabilité d'un système relationnel. Elle garantit que les relations entre les entités restent valides : une entrée dans une table liée ne peut pas exister sans sa référence dans la table d'origine.

- Dans la base de données **e_recrutement**, des clés étrangères (FOREIGN KEY) sont utilisées pour relier les commentaires et les pièces jointes aux candidatures, ainsi que les commentaires aux utilisateurs.
- Le comportement en cas de suppression est contrôlé à l'aide de la clause ON DELETE CASCADE, qui permet d'assurer qu'un enregistrement enfant est automatiquement supprimé dès que l'enregistrement parent auquel il est lié est supprimé. Cette approche garantit l'absence de données orphelines dans les tables dépendantes.

Tableau des contraintes d'intégrité référentielle		
Relation	On Delete	Description
comment.applicant_id applicant.id	Cascade	Si une candidature est supprimée, tous les commentaires associés le sont aussi.
comment.user_email user.email	Cascade	Lorsqu'un utilisateur est supprimé, ses commentaires sont automatiquement supprimés.
attachment.applicant_id applicant.id	Cascade	Supprimer une candidature supprime également tous les fichiers joints associés.

Normalisation

La **normalisation** est un processus fondamental en conception de bases de données relationnelles. Elle permet d'organiser les données pour réduire la redondance, améliorer la cohérence et faciliter la maintenance. Elle se décline en plusieurs formes normales (NF).

Première Forme Normale (1NF)

- Toutes les colonnes contiennent des valeurs atomiques (indivisibles)
- Chaque champ contient une seule valeur (pas de listes, tableaux ou valeurs répétées)
- Il n'y a aucune colonne multivaluée

La table applicant de la respecte cette règle.

```
first_name VARCHAR(50),  
last_name VARCHAR(50),  
email VARCHAR(254),
```

Chaque champ contient une valeur simple (par exemple : un seul prénom, un seul email). Il n'y a pas de liste de téléphones ou de plusieurs adresses dans une même cellule.

Deuxième forme normale (2NF)

Définition : Une table est en 2NF si elle est en 1NF et que chaque attribut non-clé dépend entièrement de la clé primaire. Cela signifie qu'il ne doit pas y avoir de dépendance partielle avec une clé primaire composite.

Exemple dans la base e_recrutement :

La table comment a une clé primaire composée de (applicant_id, user_email), et toutes les autres colonnes (comme rating, comment_body, updated_at) dépendent entièrement de cette combinaison. Aucun champ ne dépend uniquement de applicant_id ou de user_email.

Troisième forme normale (3NF)

Définition : Une table est en 3NF si elle est en 2NF et que tous les attributs non-clés ne dépendent que de la clé primaire, et pas d'autres attributs non-clés. En d'autres termes, il ne doit pas y avoir de dépendance transitive.

Exemple de non-respect dans la table applicant de la base e_recrutement :

```
zip_code VARCHAR(15),  
city VARCHAR(30),  
country VARCHAR(50)
```

Ici, zip_code permettrait déjà d'obtenir city et country, ce qui constitue une dépendance transitive. Idéalement, ces données seraient extraites dans une table location(zip_code, city, country) et référencées dans applicant.

La 3NF est utile dans des bases complexes, mais elle introduit également :

- des jointures supplémentaires dans les requêtes,
- une complexité accrue de la logique applicative (notamment dans les formulaires d'insertion),
- une dépendance au format des données de référence (ex. : fichiers postaux, pays).

Étant donné les contraintes de temps et la simplicité fonctionnelle de l'application de gestion de candidatures, le choix d'une légère dénormalisation est pertinent et pleinement justifié :

- Elle simplifie le développement et l'affichage des données (pas besoin de récupérer city via une jointure)
- Les données comme zip_code, city ou country sont saisies librement par les utilisateurs, donc peu adaptées à un modèle relationnel strict.

Il est donc pertinent de ne pas forcer la 3NF dans ce projet, car cela n'apporterait pas de réel bénéfice métier ou technique, mais augmenterait la complexité sans gain clair.

Source : <https://api-ti.pages.forge.hefr.ch/a12/db/#normalisation>

5.4. Stratégie de tests

La stratégie de tests est un ensemble de tests à effectuer pour identifier les éventuelles erreurs ou dysfonctionnements et valider le bon fonctionnement de l'application. Pour ce faire, plusieurs types de tests vont être effectués :

- Tests de compatibilité sur plusieurs navigateurs : Edge, Firefox, Chrome, Opera, Safari
- Tests de compatibilité sur plusieurs systèmes d'exploitation : MacOS, Windows
- Test de chaque fonctionnalité séparément
- Test de l'ensemble de l'application
- Beta test réalisé par des tierces personnes
- Tests des automatiques de validations (règles des différents champs) du formulaire avec Postman

Ces tests seront réalisés de manière continue tout au long du développement du projet afin d'assurer le bon fonctionnement de l'application. Le résultat des tests sera reporté dans un document en annexe.

5.4.1. Formulaire

Le tableau ci-dessous décrit les tests à effectuer pour s'assurer que la validation du formulaire fonctionne correctement.

Champ	Tests
Prénom	- Aucune valeur - Plus que 50 caractères
Nom	- Aucune valeur - Plus que 50 caractères
Date de naissance	- Aucune valeur - Format invalide (autre que YYYY.MM.DD) - Date future - Date inexistante (année bisextile)
Téléphone	- Aucune valeur - Plus que 25 caractères
Email	- Aucune valeur - Format email invalide (ex : sans @) - Plus que 254 caractères
Rue + n°	- Aucune valeur - Plus que 60 caractères
NPA	- Aucune valeur - Plus que 15 caractères

Localité	- Aucune valeur - Plus que 30 caractères
Pays	- Aucune valeur - Plus que 50 caractères
Actuellement au CO	- Aucune valeur - Type invalide (autre que booléen)
Année CO	- Aucune valeur - Type invalide (autre que les choix disponibles) - Champs rempli alors que le champ « Actuellement au CO » est désactivé
Type de classe	- Aucune valeur - Type invalide (autre que les choix disponibles) - Champs rempli alors que le champ « Actuellement au CO » est désactivé
Photo du candidat	- Aucune photo sélectionnée - Format invalide (autre que PNG, JPG, JPEG) - Photo de plus de 5 Mo - Fichier avec nom plus grand que 255 caractères
CV, lettre de motivation, bulletins scolaires, attestations de stages, autres	- Aucun fichier sélectionné - Fichier de plus de 10 Mo - Fichier avec nom plus grand que 255 caractères - Format invalide (autre que PNG, JPG, JPEG, PDF)

5.4.2. Interface administrateur

Fonctionnalités à tester sur les tailles d'écran mobile et bureau (responsive design) :

- Authentification d'un administrateur avec le compte Microsoft
- Affichage de la liste des candidatures
- Affichage des détails d'une candidature
- Affichage des commentaires dans une fenêtre overlay
- Ajouter/éditer/supprimer son commentaire
- Ajouter/éditer/supprimer le commentaire d'un autre administrateur (test de backend)
- Consulter un fichier

La protection des routes réservées aux administrateurs devra également être testée.

6. Réalisation

6.1. Structure du projet

Racine	
Fichiers/Répertoires	Description
scripts/	Contient le script SQL d'initialisation de la base de données.
postman-tests/	Contient les fichiers des tests automatiques pour Postman.
.eslintrc, .stylelintrc.json, .prettierrc.json	Fichiers de configuration pour ESLint, Stylelint et Prettier pour respecter les règles de style et formatage du code.
package.json, package-lock.json	Fichiers de configuration pour npm, liste des dépendances et les métadonnées du projet (auteur, version, etc.).
.gitignore	Liste des fichiers à exclure du contrôle de version.

Backend	
Fichiers/Répertoires	Description
package.json, package-lock.json	Fichiers de configuration pour npm, liste des dépendances et les métadonnées du projet (auteur, version, etc.).
.env.development, .env.production	Fichiers de configurations contenant les variables d'environnement.
index.js	Fichier principal qui initie et configure l'application.
middlewares/	Contient les fichiers des différents middlewares (par exemple, l'authentification, la validation du formulaire).
routes/	Contient les fichiers des différentes routes de l'API (par exemple, applicationForm.js pour la route /api/application-form).
services/	Contient les fichiers avec les fonctions pour interagir avec les différents services comme la gestion de la base de données (database.js).

utils/	Contient les utilitaires (par exemple, le logger pour la journalisation)
--------	--

Frontend	
Fichiers/Répertoires	Description
package.json, package-lock.json	Fichiers de configuration pour npm, liste des dépendances et les métadonnées du projet (auteur, version, etc.).
.env.development, .env.production	Fichiers de configurations contenant les variables d'environnement.
index.html	Point d'entrée HTML de l'application Vue.js.
vite.config.js, jsconfig.json	Fichiers de configuration pour le compilateur JavaScript et Vite.
src/components/	Contient l'ensemble des composants Vue.js.
src/18n/	Contient les fichiers de traductions (par exemple, fr.json pour les valeurs de texte de l'application en français).
src/router/	Contient la configuration du routeur Vue.js pour la navigation entre les pages.
src/services/	Contient les fichiers des différents services (par exemple, backend.js contient toutes les fonctions permettant d'envoyer des requêtes aux routes du backend).
src/App.vue	Composant principal de l'application Vue.js.
src/main.js	Fichier d'entrée principal pour initialiser l'application Vue.js.

6.2. Gestion des fichiers attachés aux candidatures

Frontend

L'interface utilisateur pour la sélection des fichiers attachés à une candidature est séparée entre une section pour la photo du candidat où le candidat peut voir le rendu de sa photo et peut la désélectionner en cliquant sur la croix du champs de sélection comme ci-dessous:

Photo du candidat

Format : PNG ou JPEG



CV, lettre de motivation, bulletins scolaires, attestations de stages, autres

Format : PDF, PNG ou JPEG



Figure 30 - Interface utilisateur pour la sélection des fichiers attachés à la candidature

La 2^e partie est un espace de sélection/drag and drop de fichiers multiples qui peuvent aussi être retiré individuellement.

Implémentation du sélecteur de photo de profil, dès que la photo est sélectionnée ou enlever, la fonction **handlePhotoChange** mets à jour l'aperçu.

```
<v-file-input
  v-model="formData.photo"
  :label="t('form.labels.photo')"
  accept="image/png, image/jpeg"
  variant="outlined"
  show-size
  :rules="rules.photoRules"
  clearable
  @update:model-value="handlePhotoChange"
/>
```

Pour les 2 champs de fichiers, un objet de type **File** est assigné à la variable de l'attribut **v-model**. Ces fichiers sont ajoutés à un objet **FormData** qui contient toutes les autres données du formulaire, ce **FormData** est ensuite envoyé dans une requête POST au backend.

Sources :

- File: <https://developer.mozilla.org/fr/docs/Web/API/File>
- FormData : <https://developer.mozilla.org/fr/docs/Web/API/FormData>

Backend

Le formulaire est réceptionné sur cette route ci-dessous dans le fichier **index.js** du répertoire **backend**, il est ensuite transmis à la route **applicationForm**.

```
app.use('/api/application-form', require('./routes/applicationForm'));
```

La route **applicationForm** ci-dessous, utilise un middleware **Multer** qui sert à séparer chaque partie du formulaire :

- Les champs de texte sont placés dans **req.body**
- Chaque fichier est placé dans **req.file** avec son nom, son type, sa taille, etc.

Sans Multer, Express ne sait pas interpréter le **multipart/form-data** ; vous recevriez donc une requête brute dont **req.body** est vide et où aucun fichier n'est accessible.

Source: <https://expressjs.com/en/resources/middleware/multer.html>

```
const multer = require('multer');
const upload = multer();

router.post('/', upload.any(), validateForm, async (req, res) => {
  try {
    const applicantId = await db.addApplicant(req.body, req.files);
    logger.info('Form submitted successfully', { applicantId });
    res.status(201).json({
      message: 'Form submitted successfully.',
      applicantId,
    });
  } catch (error) {
    logger.error('Error during form submission', { error: error.message });
    res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

La fonction **addApplicant** assure à la fois l'enregistrement des données du candidat dans la table applicant et la gestion des fichiers transmis.

Plus précisément, le cycle de vie de chaque fichier se déroule ainsi :

1. Contrôle préalable

- La présence d'une photo (fieldname === "photo") est obligatoire ; si elle manque, une exception est levée.
- Pour chaque autre fichier, l'extension est vérifiée ; seules pdf, png, jpg et jpeg sont acceptées.

2. Photo de profil

- Son contenu binaire (buffer) est directement stocké en BLOB dans la colonne photo de la table applicant lors de l'INSERT.

3. Pièces jointes (CV, bulletins, etc.)

- Un identifiant unique (UUID) est généré pour chaque fichier.
- Le fichier est écrit sur le disque (./uploads/<uuid>.<ext>) afin d'éviter toute collision de nom et d'alléger la base.
- Une entrée correspondante est ajoutée dans la table attachment avec : id, applicant_id, nom d'origine et extension.

4. Transactions et cohérence

- Toutes les opérations (insertion du candidat + écriture/référencement des pièces jointes) sont réalisées dans une même transaction SQL.
- En cas d'erreur à n'importe quelle étape, un ROLLBACK est exécuté ; rien n'est donc persistant tant que l'ensemble n'a pas abouti.

5. Journalisation et retour

- Les succès et échecs sont logués pour suivi d'audit.
- La fonction renvoie finalement l'id du candidat créé, confirmant la bonne prise en compte de toutes les données et fichiers associés.

6.3. Authentification

Ce chapitre décrit comment l'authentification via la passerelle d'authentification du CA TIC a été implémentée dans les 2 parties backend et frontend de l'application.

Frontend

Quand l'utilisateur se rend sur une page protégée et réservée aux administrateurs de l'application, le frontend suit le processus suivant décrit dans le flowchart ci-dessous.

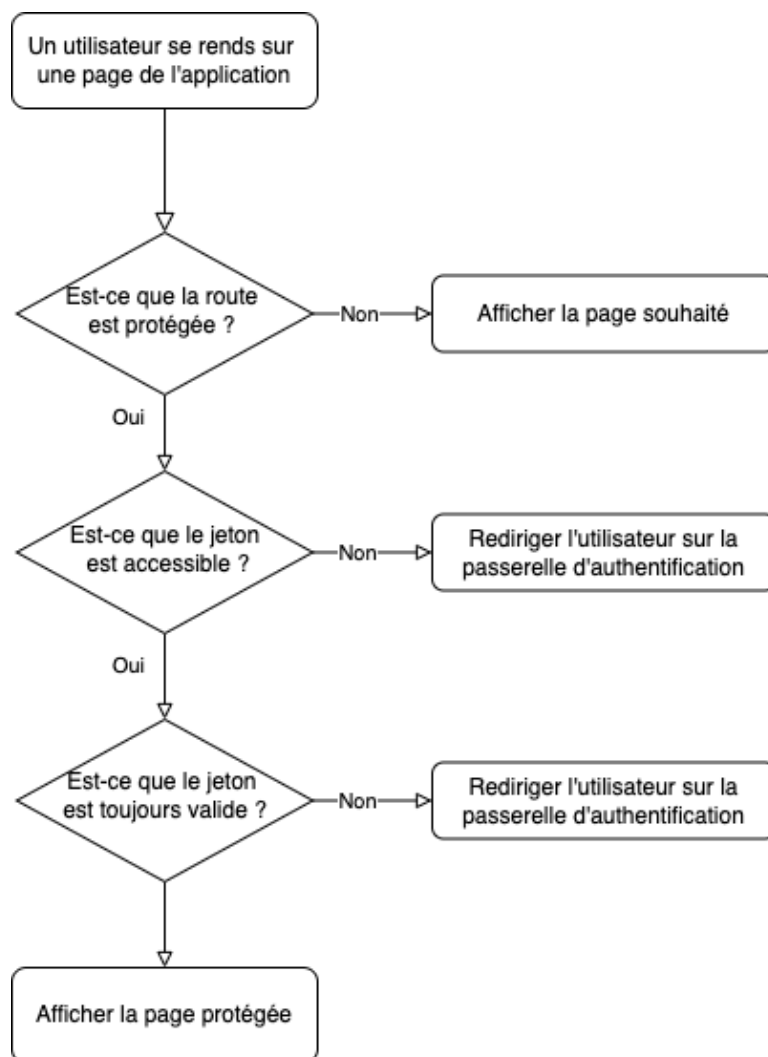


Figure 31 - Flowchart pour accéder à une page protégée

Ce processus se déroule dans le code du fichier **router.js** sur répertoire **frontend/src** :

```

router.beforeEach(async (to, from, next) => {
  // Vérifier si la route est non-protéger
  if (!to.meta.requiresAuth) return next();
  try {

```

```
await fetchToken(); // Essayer de récupérer le jeton
if (await tokenValid()) { // Vérifier la validité du jeton
  return next();
}
} catch (error) {
  await authenticateUser(); // Sinon authentifier l'utilisateur
}
});
```

Authentification d'un utilisateur

La fonction **authenticateUser** permet d'authentifier un utilisateur non-authentifié ou avec un jeton non valide, voici

```
const authenticateUser = () => {
  clearCookie(); // Enlève tout cookies liés à l'authentification
  clearAuthHeader(); // Enlève le header Authorization de l'instance axios
```

La partie suivante de la fonction redirige l'utilisateur sur la passerelle d'authentification, et passe en paramètre : **redirection_url** pour que l'utilisateur soit redirigé sur la même page à laquelle il voulait accéder initialement après le processus d'authentification.

```
window.location.href =
`${import.meta.env.VITE_GATEWAY_URL}/auth/signin?redirection_url=${encodeURIComponent(
  window.location,
)}`; //
};
```

Récupération du jeton

La fonction **fetchToken** permet de récupérer le jeton auprès du endpoint **/auth/token** de la passerelle en envoyant les données reçu lors de la redirection (cookie) après le processus d'authentification Microsoft (page de login, MFA, et ensuite redirection). Pour permettre cette opération, il faut que l'instance axios inclus le cookie dans la requête GET en activant l'option suivante :

```
const instance = axios.create({ withCredentials: true });
```

Avant d'envoyer une requête à la passerelle, la fonction vérifie d'abord si un jeton est déjà enregistré dans les cookies du client, si c'est le cas, la fonction le retourne.

Validation du jeton

Pour valider le jeton, la fonction `tokenValid` envoie une requête GET à la passerelle avec la même instance `axios` (pour avoir le jeton dans le header `Authorization`), ci-dessous l'URL de la requête :

```
`${import.meta.env.VITE_GATEWAY_URL}/auth/validity?app=${import.meta.env.VITE_GATEWAY_APP_NAME}`;
```

Exemple de réponse :

```
{
  "isValid": true,
  "familyName": "Doe",
  "givenName": "John",
  "uniqueName": "john.doe@hes-so.ch"
}
```

Envoi du jeton sur les routes protégées

Le fichier **backend.js** du répertoire **frontend/src/services** contient les fonctions permettant au frontend de communiquer avec le backend, avant chaque requête, le jeton stocké dans un cookie du navigateur du client est inclus dans le header **Authorization** de la requête :

```
instance.interceptors.request.use((config) => {
  const token = readCookie();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Backend

Pour chaque route protégée, la backend utilise un middleware **isAuthenticated**, par exemple la route `/api/admin` sera disponible seulement si le client est authentifié et validé par la passerelle d'authentification. Exemple d'utilisation dans la page `index.js` du backend :

```
app.use('/api/admin', isAuthenticated, require('./routes/admin.js'));
```

Le middleware `isAuthenticated` récupère le jeton dans le header `Authorization` de la requête et l'inclus dans la requête GET vers la passerelle d'authentification **/auth/validity**, c'est le même mécanisme que la validation du jeton du côté frontend, on fait appel à la passerelle.

6.4. Gestion des commentaires et des notes

L'application permet aux administrateurs d'ajouter un commentaire et une note pour chaque dossier de candidature. Chaque administrateur peut **ajouter, modifier ou supprimer uniquement son propre commentaire**. Il n'est pas possible de commenter plusieurs fois le même dossier.

Règles de gestion

- Un administrateur peut ajouter un seul commentaire par candidat.
- Il ne peut pas commenter à nouveau une candidature qu'il a déjà évaluée.
- Il peut modifier ou supprimer son propre commentaire.
- Il ne peut pas modifier ni supprimer les commentaires des autres.
- Une note de 1 à 5 étoiles peut être associée à un commentaire.
- Un commentaire peut avoir une note ou non, du texte ou non, mais au moins un des 2 pour pouvoir ajouter le commentaire.
- La note moyenne est calculée à partir de tous les commentaires arrondi à la valeur supérieure.

Interface utilisateur

Sur la page des détails d'une candidature, à la section des commentaires, il y a plusieurs cas possibles :

- **Cas1:** Si l'utilisateur n'a pas encore commenté, un champ de texte et un composant de notation s'affichent, par exemple:

Commentaires



☆☆☆☆☆

Ajouter un commentaire

PUBLIER

John Doe ☆☆☆☆☆

Ceci est un commentaire déjà présent de l'administrateur John Doe.

Figure 32 - Exemple d'interface pour le cas 1

- **Cas 2:** S'il a déjà commenté, son commentaire est affiché en haut de la liste, suivi des autres. Il peut cliquer sur "Modifier" pour passer son commentaire en mode édition ou sur "Supprimer" pour l'effacer.

Commentaires

Noé Gabriel

★★★★☆

Voici le commentaire de l'administrateur actuellement connecté.



John Doe

★★★★☆

Ceci est un commentaire déjà présent de l'administrateur John Doe.

Figure 33 - Exemple d'interface pour le cas 2

Sauvegarde côté backend

Dans database.js, la logique d'insertion ou de mise à jour repose sur la clé primaire composée (applicant_id, user_email) de la table comment.

- Si un commentaire existe déjà, il est mis à jour avec UPDATE.
- Sinon, un nouveau commentaire est inséré avec INSERT.

Cette vérification garantit l'unicité des commentaires par administrateur et candidat.

Lors d'un ajout, édition et suppression, le backend vérifie à quel utilisateur le jeton appartient afin que le commentaire appartient bien à l'administrateur connecté (user_email) avant d'exécuter l'opération.

6.5. Coding styles

Afin de respecter les conventions de codage du CA TIC et d'assurer une base de code homogène tout au long du projet, deux outils d'automatisation ont été mis en place, Prettier et Stylelint.

Prettier

Prettier est un outil de formatage de code source open source qui vise à rendre le code plus lisible en appliquant des conventions de formatage cohérentes et prédéfinies. Il est souvent décrit comme un "opinionated code formatter" (formateur de code avec des opinions) parce qu'il impose un style de formatage spécifique plutôt que de permettre aux développeurs de personnaliser le formatage à leur guise.

Stylelint

Stylelint est un outil d'analyse statique (linter) pour les feuilles de style. Il aide à écrire du code CSS propre et cohérent en appliquant un ensemble de règles et de conventions prédéfinies ou personnalisables. Il est largement utilisé dans le développement web pour améliorer la qualité du code CSS.

Sources :

- Conventions HTML et Prettier : <https://api-ti.pages.forge.hefr.ch/a07/code-style/>
- Conventions CSS et Stylelint : <https://api-ti.pages.forge.hefr.ch/a08/code-style/>

6.6. Validation des données du formulaire

Les données du formulaire doivent être validées 2 fois, une première fois dans le frontend lors du remplissage du formulaire et une seconde dans le backend à la réception du formulaire.

La validation côté frontend sert d'une première sécurité en cas d'oubli ou d'erreur de la part de l'utilisateur, elle sert à améliorer l'expérience utilisateur.

Cependant, cette première validation ne garantit pas la sécurité du serveur, car tout le monde peut envoyer une requête au serveur avec des données via un script ou une application comme Postman sans passer par le formulaire web.

Frontend

Pour la validation des champs du formulaire du côté client, Vuetify s'appuie sur un jeu de règles placé dans un fichier dédié et relié aux champs de formulaire via la propriété **rules** des inputs vuetify, par exemple :

```
<v-text-field
  v-model="formData.lastName"
  :label="t('form.labels.lastName')"
  variant="outlined"
  :rules="rules.lastNameRules"
  required
/>
```

Afin de mutualiser ces règles et de les réemployer aisément dans plusieurs composants, il est préférable de les regrouper dans un service distinct, puis de les importer là où l'on en a besoin. Voici un exemple de la règle du champ **lastName** dans le fichier **formValidator.js** du répertoire **frontend/services** :

```
lastNameRules: [
  (v) => !!v || t('form.rules.lastName.required'),
  (v) =>
    (v && v.length <= 50) || t('form.rules.lastName.maxLength', { max: 50
  }),
]
```

Voici un exemple du rendu qui apparaît une fois que l'utilisateur ait entré une valeur qui ne respecte pas la règle définie :



Figure 34 - Champs "Nom" avec une valeur invalide

Et si l'utilisateur envoi le formulaire sans spécifié ce champs :



Figure 35 - Champs "Nom" sans valeur

Les messages d'erreur sont également définit dans le fichier fr.json du répertoire **frontend/src/i18n** :

```
"lastName": {  
  "required": "Le nom est obligatoire",  
  "maxLength": "Le nom doit contenir au maximum {max} caractères"  
}
```

Backend

La validation des champs du formulaire côté serveur s'effectue grâce à un middleware **validateForm** défini dans le fichier **formValidator.js** du répertoire **backend/middlewares**. Le middleware fait appel à une fonction par champs, par exemple pour le champs « Nom » :

```
const validateLastName = (v) =>  
  !v  
    ? errorMsg('Last name is required.')  
    : v.length > 50  
      ? errorMsg('Last name must be 50 characters or less.')  
      : null;
```

Toutes les fonctions de validation sont dans un tableau **errors** du middleware:

```
const errors = [  
  validateFirstName(firstName),  
  validateLastName(lastName),
```

```
validateBirthDate(dateOfBirth),  
validatePhone(phone),  
validateEmail(email),  
// Etc.  
]
```

Si une erreur est relevé par le middleware, la route n'est pas accessible et un code http 400 est renvoyé au client avec la liste des erreurs.

Source : <https://developer.mozilla.org/fr/docs/Web/HTTP/Reference/Status/400>

6.7. Base de données

Dans le cadre de ce projet, la gestion des données est assurée par une base de données relationnelle MySQL. Le choix de MySQL est motivé par sa robustesse, sa performance et sa large adoption dans les applications web modernes, ainsi que sa compatibilité avec Node.js à travers la librairie mysql2/promise.

Prepared statements

L'application implémente des "prepared statements" afin d'améliorer la sécurité et la performance lors des interactions avec la base de données. Les "prepared statements" préviennent les attaques par injection SQL en séparant clairement les commandes SQL des données utilisateur.

Par exemple, lors de l'ajout d'un candidat, les données fournies sont insérées à travers une requête préparée :

```
await connection.execute(  
  `INSERT INTO applicant (id, first_name, last_name, ...) VALUES (?, ?, ?,  
  ...)` ,  
  [id, firstName, lastName, ...]  
);
```

Source : <https://sidorares.github.io/node-mysql2/docs/documentation/prepared-statements>

Transactions

L'intégrité des données est primordiale. Afin de garantir la cohérence des données, l'application utilise des transactions pour les opérations impliquant plusieurs étapes qui doivent toutes réussir ou toutes échouer ensemble. Par exemple, lors de l'ajout d'un candidat, l'insertion du candidat ainsi que celle des pièces jointes sont encapsulées dans une transaction unique :

```
await connection.beginTransaction();
```

```
try {  
  // Insertion candidat  
  await connection.execute(...);  
  
  // Insertion pièces jointes  
  for (const file of files) {  
    if (file.fieldname !== 'photo') {  
      await connection.execute(...);  
    }  
  }  
  
  await connection.commit();  
} catch (error) {  
  await connection.rollback();  
  throw error;  
}
```

Si une erreur se produit, la transaction est annulée, et toutes les modifications apportées durant la transaction sont annulées, cela garantit que soit toutes les données liées à un candidat sont insérées, soit aucune en cas d'erreur.

En résumé, le choix et l'utilisation judicieuse de MySQL, accompagnés de la librairie mysql2/promise, de prepared statements et de transactions, assurent une gestion efficace, sécurisée et fiable des données pour l'ensemble de l'application.

6.8. Traitement des erreurs

Frontend

Dans l'application frontend, toutes les opérations asynchrones susceptibles de provoquer une erreur (appel API, chargement de données, envoi de formulaire, etc.) sont encapsulées dans des blocs try...catch. Cette structure permet de capturer les erreurs et d'afficher un message explicite à l'utilisateur via des composants v-alert de Vuetify.

Exemple typique dans le service backend.js :

```
const getApplicantsList = async () => {  
  try {  
    const response = await instance.get('/admin/applicants-list');  
    return response.data;  
  }  
}
```

```
} catch (error) {  
  throw error; // propagé au composant Vue pour affichage  
}  
};
```

Dans le composant Vue ApplicantsList.vue, ces erreurs sont capturées et un message d'erreur est affiché :

```
try {  
  applicants.value = await backend.getApplicantsList();  
} catch {  
  error.value = t('admin.errors.fetchApplicants');  
}
```

L'interface est ainsi capable d'informer clairement l'utilisateur en cas d'échec, tout en restant fonctionnelle, par exemple:

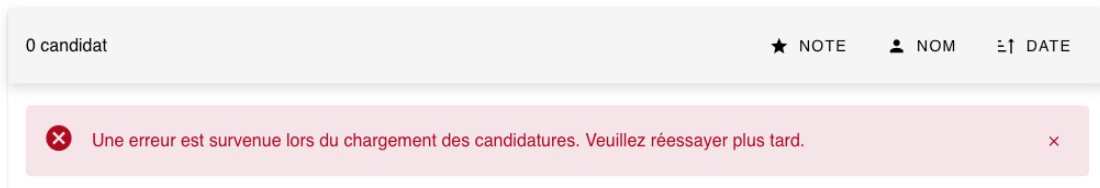


Figure 36 - Exemple affichage d'une erreur

Backend

Le backend repose sur Express.js. La gestion des erreurs est centralisée dans les routes à l'aide de blocs try...catch. En cas d'erreur, une réponse JSON avec un code HTTP approprié est renvoyée.

Exemple :

```
router.get('/applicant/:id', async (req, res) => {  
  try {  
    const applicant = await getApplicantDetails(id);  
    res.json(applicant);  
  } catch (error) {  
    if (error.message === 'Applicant not found') {  
      return res.status(404).json({ error: 'Applicant not found' });  
    }  
  }  
}
```

```
logger.error('Error fetching candidate details', { error: error.message
});
res.status(500).json({ error: 'Internal server error' });
}
});
```

Les erreurs sont différenciées selon leur nature (erreur utilisateur, ressource manquante, ou erreur système) et les statuts HTTP sont respectés (400, 401, 404, 500...).

Journalisation

La journalisation est exclusivement gérée côté backend à l'aide de la bibliothèque winston combinée à winston-daily-rotate-file, ce qui permet :

- Une rotation quotidienne des fichiers de logs
- Une séparation des erreurs critiques (error.log) et des logs combinés (combined.log)
- Un archivage automatique et limité dans le temps

Source : <https://github.com/winstonjs/winston>

Extrait de configuration (logger.js) :

```
const logger = createLogger({
  level: 'info',
  format: combine(timestamp(), prettyPrint()),
  transports: [
    new DailyRotateFile({
      filename: `${process.env.LOG_PATH}error-%DATE%.log`,
      level: 'error',
      ...
    }),
    new DailyRotateFile({
      filename: `${process.env.LOG_PATH}combined-%DATE%.log`,
      ...
    }),
  ],
});
```

Tous les événements critiques comme les erreurs de base de données, les échecs d'authentification, ou les exceptions applicatives sont enregistrés, permettant un audit précis et une meilleure traçabilité en production.

6.9. Style et design avec Vuetify

Vuetify est une bibliothèque d'interface utilisateur pensée pour Vue.js, elle propose un vaste catalogue de composants prêts à l'emploi qui simplifient la création d'interfaces ergonomiques.

Material Design	L'utilisation des principes Material Design assure une cohérence visuelle sur l'ensemble des écrans.
Compatibilité navigateur étendue	Fonctionne sans problème sur la plupart des navigateurs modernes, y compris ceux basés sur Chromium (Chrome, Opera, Edge) ainsi que Firefox et Safari
CSS Reset	Normalise les styles par défaut des éléments HTML, ce qui réduit les divergences d'affichage entre navigateurs.
Responsive design	L'ensemble des composants est adaptable aux différentes tailles d'écran

Dans le contexte de ce projet, le temps est limité et un des besoins mentionné dans le cahier des charges est de développer une application moderne, de ce fait, il est pertinent d'utiliser une librairie de composants graphique déjà « stylisé », pour ne pas avoir besoin de d'allouer du temps sur des détails comme par exemple, la taille, la couleur et l'animation d'un bouton.

Sources : <https://vuetifyjs.com>, <https://m3.material.io>

6.10. Gestion des langues

L'application doit être en français, mais doit permettre une évolution vers une application en différentes langues, c'est l'internationalisation.

Pour ce faire, il faut développer l'application sur une base qui permettra cette évolution, dans l'écosystème Vue.js, on utilise couramment le plugin **Vue I18n** pour gérer cette fonctionnalité.

Les traductions françaises sont stockées dans le fichier **fr.json** placé dans le dossier **i18n** du répertoire **frontend/src**, par la suite il pourra aussi y avoir le fichier en.json, de.json, it.json, etc.

Dans les templates, on accède aux chaînes traduites avec la fonction **t** suivie de la clé JSON, par exemple, le code ci-dessous permet d'afficher le label « Type de classe » au dessous des boutons radio dans le formulaire.

```
<label>{{ t('form.labels.classType') }}</label>
```

On peut également accéder à ces traductions dans les scripts js.

Source : <https://vue-i18n.intlify.dev>

6.11. Gestion des configurations

Pour éviter de disperser les valeurs sensibles ou spécifiques à chaque contexte d'exécution, toutes les configurations sont centralisées dans des fichiers `.env` sous forme de paires **clé = valeur**, par exemple :

```
UPLOAD_ALLOWED_ATTACHMENT_TYPES=application/pdf,image/png,image/jpeg
```

Dans un projet Vue, les variables sont exposées automatiquement via **import.meta.env**.

Attention : seules celles dont le nom commence par `VITE_` sont injectées dans le code client.

```
const variable = import.meta.env.VARIABLE;           // Ne fonctionne pas
const variable = import.meta.env.VITE_VARIABLE;       // Fonctionne
```

Source : <https://vite.dev/guide/env-and-mode>

Par contre, dans un projet Express, les variables du fichier `.env` sont accessibles sans préfixe et il faut utiliser une librairie **dotenv**, par exemple :

```
require('dotenv').config();
console.log(process.env.VARIABLE);
```

Source : <https://www.npmjs.com/package/dotenv>

Gestion des modes

Il est courant d'utiliser des réglages différents selon l'environnement. Pour cela, on crée 2 fichiers **.env.development** et **.env.production**.

Pour Vue:

- **npm run dev** charge `.env.development`.
- **npm run build** charge `.env.production`.

Pour Express:

- **NODE_ENV=production npm start** chargera `.env.production`.
- **NODE_ENV=development npm start** utilisera `.env.development`.

7. Tests de fonctionnement

Tests Postman de validation des champs du formulaire

Un fichier **eRecrutement-form-validation-tests.postman_collection.json** dans le répertoire **postman-tests/** permet d'exécuter la commande suivante :

```
newman run eRecrutement-form-validation-tests.postman_collection.json
```

Cette commande exécute **35 tests** automatiques visant à valider les différentes règles de validation du formulaire de postulation, aussi bien pour les champs de saisie que pour les fichiers joints (taille maximale, type MIME, etc.).

Les tests couvrent notamment :

- Les valeurs obligatoires
- Les formats attendus (dates, emails, numéros)
- Les longueurs maximales
- Les valeurs autorisées
- La validation des fichiers (présence, format, taille, poids total)

Tous les tests ont été passés avec succès comme l'indique le rapport suivant :

	executed	failed
iterations	1	0
requests	29	0
test-scripts	29	0
prerequisite-scripts	0	0
assertions	58	0
total run duration: 887ms		
total data received: 1.39kB (approx)		
average response time: 15ms [min: 2ms, max: 108ms, s.d.: 27ms]		

Figure 37 - Rapport de tests Newman

Rapport de test

Un rapport de test détaillé a été réalisé afin de valider le bon fonctionnement de l'application. Il est joint en annexe sous forme de fichier Excel nommé **Tests_2025_151928_GabrielNoe.xlsx**. Ce document contient l'ensemble des cas de test effectués, classés par fonctionnalité (frontend et backend), et documente pour chaque test :

- Le numéro du test (F pour frontend, B pour backend, exemple : F1, F2, F3, etc.)
- La procédure pas à pas
- Le résultat attendu
- Le résultat obtenu
- La validation
- Les remarques éventuelles

Test utilisateur

Yoan Gilliland, étudiant en ingénierie logicielle à la HEIA-FR :

« Je trouve que l'application est bien pensée et facile à utiliser. L'interface est claire, les étapes pour postuler sont bien guidées, et on s'y retrouve rapidement. C'est un outil pratique qui pourrait réellement simplifier les démarches de recrutement dans les écoles et les entreprises formatrices.

J'ai également pu tester l'interface d'administration, que j'ai trouvée très simple et intuitive. Elle permet de gérer les candidatures sans difficulté, avec une bonne organisation des informations. C'est un vrai plus pour les responsables du recrutement. »

8. Problème rencontré

Validation des entrées de fichiers

Lors des tests de validation des champs du formulaire, un problème a été rencontré concernant l'automatisation de l'upload de fichiers avec l'application de bureau Postman. Bien que celle-ci permette d'automatiser les tests pour les champs textuels, elle ne prend pas en charge l'automatisation des champs de type fichier.

Alternative trouvée : Newman, une application en ligne de commande similaire à Postman, permet d'automatiser ce type de test. Les tests de validation pour les champs de fichiers ont ainsi pu être réalisés avec succès à l'aide de cette solution.

Source : <https://learning.postman.com/docs/collections/using-newman-cli/newman-file-uploads/>

9. Amélioration et évolution

Il y a plusieurs améliorations possibles pour l'application eRecrutement, ce chapitre décrit quelques possibilités futures.

Filtrer les candidatures par années d'entrée en fonction

Cette fonctionnalité inclut les points suivants :

- Sur la page du formulaire : Indiquer dynamiquement la date d'entrée en fonction.
- Sur la page de la liste des candidatures : Un champ de type sélecteur pour filtrer les candidatures selon leur date d'entrée en fonction.

Par exemple : Afficher uniquement les candidatures pour la place d'apprentissage qui commence en août 2026 ou afficher toutes les candidatures pour la place ayant débuté en août 2022, etc.

Cette possibilité a été discutée avec le chef de projet. La réalisation de l'application a pris en compte cette évolution potentielle en enregistrant automatiquement la date et l'heure auxquelles chaque dossier de candidature est soumis. Cette prévision facilite grandement l'implémentation future de cette fonctionnalité.

Système d'archivage des candidatures

Un système automatisé d'archivage pourrait être implémenté pour gérer efficacement les dossiers de candidatures sur le long terme. Cette fonctionnalité consisterait à :

- Définir une période après laquelle les candidatures seront automatiquement archivées (par exemple, après 6 ou 12 mois).
- Stocker les candidatures archivées dans une base de données ou un espace de stockage dédié, afin de réduire la charge de données actives.
- Permettre un accès facilité et sécurisé aux dossiers archivés à travers l'interface administrateur, avec des capacités de restauration ou consultation en cas de besoin.

Solution d'anti-spam pour le formulaire de postulation

Afin d'améliorer la sécurité et la qualité des candidatures soumises via le formulaire en ligne, il serait judicieux d'intégrer une solution d'anti-spam, comme Google reCAPTCHA ou une solution similaire. Cela permettrait :

- D'éviter les soumissions automatisées par des bots.
- D'améliorer la qualité des dossiers reçus en éliminant les spams.
- De renforcer la sécurité globale du formulaire en limitant les risques de saturation du système par des requêtes malveillantes ou inutiles.

Désarmement et reconstruction du contenu (CDR)

Le désarmement et la reconstruction du contenu (CDR) est une mesure avancée de sécurité, visant à :

- Scanner les documents et fichiers soumis par les utilisateurs afin d'éliminer les contenus potentiellement dangereux.
- Reconstruire les fichiers en ne conservant que le contenu validé, sans éléments nuisibles ou non désirés (par exemple : macros, scripts malveillants).

10. Conclusion

Technique

L'application eRecrutement est fonctionnelle et réponds aux besoins spécifiques du Centre d'Apprentissage TIC de la HEIA-FR en matière de recrutement d'apprentis. Toutes les fonctionnalités définies dans le cahier des charges ont été implémentées, testées et validées avec succès. Les candidats peuvent désormais remplir un formulaire intuitif et responsive, incluant la soumission sécurisée de fichiers. Les recruteurs disposent d'une interface claire leur permettant de consulter les candidatures, d'ajouter des notes et commentaires, de trier les dossiers et de consulter les fichiers joints. L'intégration de l'authentification centralisée via la CA TIC Authentication Gateway assure une sécurité renforcée et un contrôle d'accès efficace.

Personnelle

Ce projet m'a offert une immersion complète dans le développement d'une application professionnelle, de la phase d'analyse jusqu'à la phase de test. J'ai pu appliquer et approfondir mes connaissances en Node.js, Vue.js, Express, SQL, ainsi qu'en sécurité web et en architecture logicielle. La diversité des tâches (authentification, gestion des fichiers, conception de base de données, mise en place d'une interface utilisateur responsive) m'a permis d'acquérir une vision globale d'un projet complet, tout en renforçant ma rigueur dans le développement et la documentation. Ce travail m'a également permis de prendre conscience de l'importance de la planification, de l'anticipation des besoins futurs, et de la gestion des imprévus techniques.

11. Remerciements

Je tiens à remercier chaleureusement mon responsable, M. Joël Dacomo pour son accompagnement tout au long de ces années. Ses conseils, son exigence de rigueur et sa disponibilité ont été déterminants pour la réalisation de ce projet et pour ma progression professionnelle.

Je remercie également M. Mario Ramalho et M. Marc Stritt pour leur encadrement lors de leurs deux visites à la Haute École d'Ingénierie et d'Architecture de Fribourg.

Enfin, j'adresse mes remerciements à Yoan Gilliand, ancien apprenti et étudiant en ingénierie logicielle, pour sa disponibilité durant les phases de test, ainsi que pour ses retours précieux en tant qu'utilisateur de l'interface.

12. Déclaration de non plagiat

Je soussigné Noé Mathys Gabriel déclare que les informations contenues dans ce rapport de travail pratique individuel rendu ce jour le 20.05.2025 dans le cadre de la procédure de qualification de mon CFC d'informaticien, ne sont pas plagiées. Toutes les informations de sources extérieures ainsi que les informations fournies par des tiers durant le déroulement du travail sont consignées.

Date	Nom et prénom	Signature
20 mai 2025	Gabriel Noé Mathys	

13. Glossaire

Glossaire	
Abréviation	Description
API	Application Programming Interface
Backend	Partie d'une application qui gère la logique métier, les bases de données, les règles de sécurité et la communication avec le frontend.
CFC	Certificat Fédéral de Capacité. Diplôme professionnel suisse obtenu à la fin d'une formation en apprentissage.
CO	Cycle d'Orientation
CA TIC	Centre d'Apprentissage TIC
Frontend	Partie visible d'une application web, affichée dans le navigateur et avec laquelle l'utilisateur interagit.
HTTP	Hypertext Transfer Protocol. Protocole utilisé pour la communication sur le web.
HTTPS	Version sécurisée du protocole HTTP utilisant SSL/TLS pour chiffrer les échanges.
HEIA-FR	Haute École d'Ingénierie et d'Architecture de Fribourg
JWT	JSON Web Token
JPG	Joint Photographic Experts Group
MFA	Multi-Factor Authentication
Microsoft Entra	Plateforme de gestion des identités et des accès cloud proposée par Microsoft (anciennement Azure AD).
Node.js	Environnement d'exécution JavaScript côté serveur permettant de créer des applications web.
PDF	Portable Document Format
PNG	Portable Network Graphics
REST / RESTful	Representational State Transfer

SQL	Structured Query Language
SSO	Single Sign-On
UUID	Universally Unique Identifier
UX	Expérience utilisateur
Vue.js	Framework JavaScript progressif pour la création d'interfaces utilisateur dynamiques et réactives.
Vuetify	Framework UI basé sur Vue.js permettant de créer des interfaces avec le design Material Design.
XSS	Cross-Site Scripting

14. Annexes

L'annexe « TPI_2025_151928_GabrielNoe » est structurée comme suit :

Annexe	Description
CDC	Cahier des charges au format PDF
Code	Contient le code de l'application
Journal	Journal de bord du projet au format PDF
Planification	Planification du projet au format PDF
PV	Contient les différents PV des séances au format PDF
Séquences	Contient les UML de séquences au format PNG
Tests	Rapport de tests au format XLSX
CollectionTestsPostman	Collection de tests Postman

15. Table des illustrations

Figure 1 - Planification du 7 au 13 mai	7
Figure 2 - Planification du 14 au 20 mai	7
Figure 3 - Cahier des charges: Mockup de formulaire de postulation.....	9
Figure 4 - Cahier des charges : Mockup de la liste des candidatures	10
Figure 5 - Cahier des charges: Mockup de la fenêtre des commentaires	11
Figure 6 - Cahier des charges: Mockup de la page de détails d'une candidature.....	12
Figure 7 - Principe de fonctionnement.....	13
Figure 8 - Représentation UML d'un acteur.....	14
Figure 9 - Représentation UML d'un cas d'utilisation	14
Figure 10 - Représentation UML d'une association	14
Figure 11 - Représentation UML d'une inclusion d'un cas d'utilisation	14
Figure 12 - diagramme de cas d'utilisation	15
Figure 13 - Ébauche conceptuelle: Authentification	26
Figure 14 - Ébauche conceptuelle: Envoi d'une candidature.....	27
Figure 15 - Ébauche conceptuelle: Affichage de la liste des candidatures	27
Figure 16 - Ébauche conceptuelle: Affichage des détails d'une candidature	27
Figure 17 - Ébauche conceptuelle: Ajouter, modifier, supprimer un commentaire et note.....	28
Figure 18 - Ébauche conceptuelle: Consulter les fichiers d'une candidature	29
Figure 19 - Schéma d'architecture 3-tiers.....	34
Figure 20 - Diagramme de séquence: Authentification	35
Figure 21 - Diagramme de séquence: Envoi d'une candidature	36
Figure 22 - Diagramme de séquence: Affichage de la liste des candidatures	37
Figure 23 - Diagramme de séquence: Affichage des détails d'une candidature	38
Figure 24 - Diagramme de séquence: Affichage des commentaires d'une candidatures	39
Figure 25 - Diagramme de séquence: Ajout d'un commentaire.....	40
Figure 26 - Diagramme de séquence: Modification d'un commentaire.....	41
Figure 27 - Diagramme de séquence: Suppression d'un commentaire.....	42
Figure 28 - Diagramme de séquence: Consulter le fichier d'une candidature	43
Figure 29 - Schéma de la base de données.....	44
Figure 30 - Interface utilisateur pour la sélection des fichiers attachés à la candidature	54
Figure 31 - Flowchart pour accéder à une page protégée	58
Figure 32 - Exemple d'interface pour le cas 1	61
Figure 33 - Exemple d'interface pour le cas 2	62
Figure 34 - Champs "Nom" avec une valeur invalide.....	64
Figure 35 - Champs "Nom" sans valeur	64
Figure 36 - Exemple affichage d'une erreur	67
Figure 37 - Rapport de tests Newman.....	71